

TÀI LIỆU KHÓA HỌC

Java Cơ Bản (Java Core)

Tác giả: Hỏi Dân IT & Eric

Version: 1.0

Hướng Dẫn Sử Dụng Udemy Dành Cho Người Mới Bắt Đầu	5
Cách Hỗ Trợ/Support Trong Quá Trình Học (Bắt Buộc Xem)	6
Chapter 1: Bắt buộc xem - Không bỏ qua chương học này	9
#1. Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả	9
#2 + #3 + #4 : Hướng Dẫn Truy Cập Tài Liệu Khóa Học	11
#5. Demo kết quả đạt được	12
#6. Về Quyền Tác Giả	13
#7 + #8. Thông Tin Tác Giả Hỏi Dân IT	14
Chapter 2: Giới thiệu về Java	15
#9. Tổng quan về các kiến thức sẽ học	15
#10. Java là gì ?	16
#11. Phân biệt JVM, JRE, JDK	19
#12. Chuyện Cài Đặt Công Cụ (Bắt Buộc Xem)	21
#13. Cài Đặt Môi Trường Java	22
#14. Cài Đặt Công Cụ Coding Java	23
#15. Hello World với Java	26
#16. Quá trình thực thi Java (Extra)	27
#17. Sử dụng Git để quản lý mã nguồn	28
Chapter 3: Cú pháp cơ bản và biến	29
#18. Tổng quan về chapter	29
#19. Variables (Khái niệm biến)	29
#20. Kiểu Dữ Liệu Nguyên Thủy (Primitive Types)	31
#21. Kiểu Dữ Liệu Tham Chiếu (Reference Types)	33
#22. Boxing và Unboxing	35
#23. Toán Tử Trong Java	37
#24. Nhập và Xuất Dữ Liệu Trong Java	38
#25. Bài Tập Lab 01	41
#26. Chữa Bài Tập Lab 01	42
Chapter 4: Cấu trúc điều khiển	43
#27. Tổng quan về chapter	43
#28. Câu lệnh điều kiện - if, if-else, if-else if	43
#29. Bài Tập Câu Lệnh Điều Kiện	44
#30. Câu lệnh switch-case	45
#31. Bài Tập Câu Lệnh Switch-Case	47
#32. Vòng lặp for, while, do-while	48
#33. Bài Tập Sử Dụng Vòng Lặp (Loop)	50
#34. Câu lệnh điều khiển luồng – break, continue, return	51
#35. Bài Tập Câu Lệnh Điều Khiển Luồng (Break/Continue)	52
#36. Bài Tập Lab 02	53
#37. Chữa Bài Tập Lab 02	54
Chapter 5: Mảng và Chuỗi	55
#38. Tổng quan về chapter	55
#39. Mảng 1 Chiều – Cấu trúc, Khởi tạo, Duyệt mảng (Array)	55

#40. Bài Tập Mảng 1 Chiều	56
#41. Giới Thiệu Mảng Đa Chiều (Extra)	57
#42. Vòng lặp for-each trong Java	59
#43. Bài Tập for-each	60
#44. Lớp String & Các Phương Thức Phổ Biến	61
#45. So sánh chuỗi == vs equals()	62
#46. Xử lý chuỗi hiệu quả với StringBuilder & StringBuffer (Extra)	63
#47. Bài Tập Lab 03	66
#48. Chữa Bài Tập Lab 03	67
Chapter 6: Lập trình hướng đối tượng cơ bản (OOP)	68
#49. Tổng quan về chapter 6	68
#50. Lập trình hướng đối tượng (Object Oriented Programming - OOP) là gì ?	68
#51. Khái niệm Class và Object	69
#52. Bài Tập tạo Class/Object	71
#52.1 Hiện thị Package Collapse (Extra)	72
#53. Thuộc tính và Phương thức (Fields & Methods)	73
#54. Bài Tập Fields/Methods	75
#55. Constructor – Hàm tạo trong Java	76
#56. Bài Tập Constructor (Hàm tạo)	77
#57. Package và Access Modifier	78
#58. Bài Tập Access Modifier	80
#59. Encapsulation – Tính đóng gói trong Java	81
#60. Bài Tập Lab 04	82
#61. Chữa Bài Tập Lab 04	84
Chapter 7: Kế thừa (Inheritance)	85
#62. Tổng quan về chapter 7	85
#63. Kế thừa và từ khóa extends	85
#64. Bài Tập Kế Thừa	86
#65. Từ khóa super trong constructor và phương thức	87
#66. Bài Tập sử dụng Super	88
#67. Ghi đè phương thức (Method Overriding)	89
#68. Bài Tập Method Overriding	90
#69. Nạp chồng phương thức (Method Overloading)	91
#70. Bài Tập Method Overloading	93
#71. Bài Tập Lab 05	94
#72. Chữa Bài Tập Lab 05	95
Chapter 8: Đa hình (Polymorphism) và Trừu tượng (Abstraction)	96
#73. Tổng quan về chapter 8	96
#74. Phân biệt Compile-time vs Runtime (Extra)	96
#75. Tính Đa Hình (Polymorphism)	98
#76. Ứng dụng đa hình thực tế – Upcasting, Downcasting, instanceof	99
#77. Method Resolution của Java (Extra)	101
#78. Bài Tập về tính Đa Hình	102
#79. Abstract Class	103

#80. Bài Tập Abstract Class	105
#81. Interface	107
#82. Bài Tập Interface	108
#83. Abstract Class vs Interface (Extra)	109
#84. Bài Tập Lab 06	111
#85. Chữa Bài Tập Lab 06	112
Chapter 9: Enum và Nested Class	113
#86. Tổng quan về chapter 9	113
#87. Từ khóa final và static (Extra)	113
#88. Sử dụng enum thay thế static final	115
#89. Thêm thuộc tính và phương thức vào enum	116
#90. Bài Tập về enum	117
#91. Inner Class và Static Nested Class	118
#92. Local Class và Anonymous Class	120
#93. Bài Tập Lab 07	122
#94. Chữa Bài Tập Lab 07	124
#95. Nhìn lại các kiến thức đã học	124
Chapter 10: Xử lý ngoại lệ (Exception)	125
#96. Tổng quan về chapter 10	125
#97. Exception và Try/Catch/Finally	125
#98. Java - Try with Resources (Extra)	127
#99. Sử dụng throw và throws trong xử lý lỗi	127
#100. Bài Tập Try/Catch/Throw	129
#101. Phân Loại Exception (Extra)	130
#102. Tạo Custom Exception	132
#103. Về Xử Lý Exception	133
#104. Bài Tập Lab 08	134
#105. Chữa Bài Tập Lab 08	136
Chapter 11: Collection Framework	137
#106. Tổng quan về chapter 11	137
#107. Big O là gì (Extra)	137
#108. Tổng quan về Collection Framework	139
#109. List và ArrayList	141
#110. LinkedList (Extra)	142
#111. Bài Tập về ArrayList	143
#112. Set, HashSet và TreeSet	145
#113. Quá trình thêm phần tử vào HashSet/HashMap (Extra)	147
#114. Bài Tập HashSet	149
#115. Map, HashMap và TreeMap	150
#116. Iterator và ListIterator	152
#117. Bài Tập Lab 09	155
#118. Chữa Bài Tập Lab 09	156
Chapter 12: Generics trong Java	157
#119. Tổng quan về chapter 12	157

#120. Generics là gì ?	157
#121. Generic Class và Generic Method	159
#122. Bài Tập về Generics	161
#123. Unbounded Wildcard (<?>) - Wildcard không giới hạn	162
#124. Upper Bounded Wildcard (<? extends T>) - Wildcard giới hạn trên	164
#125. Lower Bounded Wildcard (<? super T>) - Wildcard giới hạn dưới	165
#126. Type Erasure (Extra)	166
#127. Bài Tập Lab 10	167
#128. Chữa Bài Tập Lab 10	168
Chapter 13: Java I/O (Input/Output)	169
#129. Tổng quan về chapter 13	169
#130. Phân Loại Đọc/Ghi File ?	169
#131. Đọc/Ghi file với FileReader/FileWriter/PrintWriter	171
#132. Đọc/Ghi file với BufferedReader/BufferedWriter	173
#133. Sử dụng java.nio.file.Files (Character Stream)	175
#134. Bài Tập Đọc/Ghi File (Character Streams)	176
#135. Đọc/Ghi file với FileInputStream & FileOutputStream	178
#136. Đọc/Ghi file với BufferedInputStream & BufferedOutputStream	179
#137. Sử dụng java.nio.file.Files (Byte Stream)	179
#138. Bài Tập Đọc/Ghi File (Byte Stream)	180
#139. Serialization & Serializable Interface (Extra)	181
#140. serialVersionUID & Warning khi extends Exception	182
Chapter 14: Đa luồng trong Java	183
#141. Tổng quan về chapter 14	183
#142. Giới thiệu Đa luồng (Thread)	183
#143. Tạo Thread (extends Thread)	185
#144. Tạo Thread (implements Runnable)	186
#145. Vòng đời của Thread (Thread Lifecycle)	187
What's next ? Học gì tiếp theo	188
Đánh Giá (Review/Rating) Khóa Học	190

Hướng Dẫn Sử Dụng Udemy dành Cho Người Mới Bắt Đầu

Tài liệu hướng dẫn chi tiết xem [tại đây](#)

Mục tiêu: hướng dẫn sử dụng các tính năng chủ yếu của Udemy, hỗ trợ việc học tập hiệu quả hơn, dành cho người lần đầu tiên sử dụng nền tảng Udemy.

1. Các tính năng cơ bản

- Hiển thị giao diện ngôn ngữ tiếng anh/tiếng việt
- Khóa học của tôi nằm ở đâu ? (my learning - kiểm tra các tabs)
- Trình phát video
- Điều chỉnh chất lượng, tốc độ video
- Download tài liệu (nếu có)

2. Các tính năng khác

- Bạn có thể sử dụng Udemy App trên [mobile](#). Download [tại đây](#)
- Q/A, Notes, Announcements

Cách Hỗ Trợ/Support Trong Quá Trình Học (Bắt Buộc Xem)

Mục đích:

- **Giải đáp thắc mắc** của học viên liên quan về nội dung khóa học
- **Hỗ trợ gỡ lỗi** (fix bug) trong quá trình thực hành theo video hướng dẫn

1. Tính năng Q&A (Hỏi/Đáp)

Sử dụng tính năng này khi bạn cần giải đáp thắc mắc & hỗ trợ trong quá trình thực hành, với các ưu điểm vượt trội sau:

- **Bảo vệ sự riêng tư của bạn** (chỉ hiển thị tên tài khoản Udemy)
- **Hỗ trợ upload hình ảnh, gõ code...** giao diện trực quan, dễ theo dõi, tránh tình trạng trôi tin nhắn (so với cách chat text)
- **Hỗ trợ lọc/filter câu hỏi, xem lịch sử** các câu hỏi đã đặt ứng với video đang xem
- **Cung cấp ngữ cảnh** (gặp vấn đề tại video nào), như vậy nó sẽ rõ ràng & giải quyết nhanh hơn

2. Hướng dẫn cách đặt câu hỏi

Trước khi đặt câu hỏi, bạn nên lọc câu hỏi theo video, mục đích là **kiểm tra vấn đề bạn hỏi, đã có người khác hỏi hay chưa**, giúp bạn tiết kiệm thời gian hơn.

Các nguyên tắc khi đặt câu hỏi:

- Đặt câu hỏi theo video bạn thắc mắc/cần hỗ trợ
- Câu hỏi cần rõ ràng, tường minh, tránh nêu vấn đề chung chung, không cụ thể.

Minh họa cách đặt câu hỏi:

Bước 1: Chọn bài giảng bạn “gặp vấn đề” cần hỗ trợ

Bước 2: Duyệt qua danh sách câu hỏi ứng với video đang xem (nếu có), vì nếu như đã có vấn đề bạn gặp phải, sẽ giúp bạn tiết kiệm thời gian.

Bước 3: Đặt câu hỏi mới

- **Đặt tiêu đề:** nếu ngắn gọn (đặt tên) cho vấn đề bạn muốn hỏi
- **Viết nội dung** chi tiết: miêu tả cụ thể vấn đề gặp phải

Hướng dẫn: chèn **link** (ví dụ google drive), chèn **ảnh** và gõ **code**.

Sửa (update) câu hỏi nếu cần thiết.

Gợi ý các mẫu đặt câu hỏi:

Mẫu 1 – Khi gặp lỗi code:

Tôi đang học tới **[đoạn nào trong video]**, gặp vấn đề **[Mô tả ngắn về lỗi]**.

Tôi đã thử **[Các bước đã làm để xử lý]**, nhưng kết quả là **[Thông báo lỗi hoặc tình trạng xảy ra]**.

Đây là đoạn code/hình ảnh minh họa:

[Đoạn code ngắn hoặc ảnh chụp màn hình]

Mẫu 2 - Hỏi kỹ năng:

Trong bài **[Tên bài hoặc chủ đề]**, tôi thấy phương pháp/kỹ thuật **[Tên phương pháp]** rất hay, nhưng chưa rõ cách áp dụng trong **[Ngữ cảnh thực tế của bản thân]**.

Tôi mong giảng viên/anh chị có thể cho thêm một ví dụ minh họa cụ thể để tôi dễ hình dung hơn.

3. Hướng dẫn cách xem câu trả lời

Udemy sẽ hiển thị thông báo trong tài khoản Udemy, và gửi email khi có phản hồi.

Thông thường, thời gian phản hồi câu hỏi là 24h (ngày lễ, thứ 7, chủ nhật có thể lâu hơn).
Tối đa thời gian nhận phản hồi là 02 ngày làm việc (2 business days)

Khi mở email, cần kiểm tra từ inbox (primary), update, social, spam...

//todo: hướng dẫn kéo thả email vào primary (**no-reply@e.udemymail.com**)

(Mỗi lần enroll, sẽ có email tới từ: **hello@alerts.udemy.com**)

4. Hướng dẫn cách nhận thông báo (nếu có)

Cấu hình [tại đây](#)

5. Kênh liên hệ giảng viên trên Udemy (message)

Profile giảng viên: <https://www.udemy.com/user/eric-7039/>

Chapter 1: Bắt buộc xem - Không bỏ qua chương học này

Hướng dẫn sử dụng khóa học hiệu quả, đạt chất lượng cao nhất

#1. Hướng Dẫn Sử Dụng Khóa Học Hiệu Quả

Bạn vui lòng "xem video lần lượt" theo trình tự. Vì khóa học như 1 dòng chảy, video sau sẽ kế thừa lại kết quả của video trước đó.

1. Dành cho học viên "có ít thời gian"

Nếu bạn vội, cần học nhanh, hoặc "bạn đã biết rồi", thì "vẫn xem video, cơ mà không cần code theo".

Lưu ý: vẫn xem qua tài liệu khóa học để biết "video hướng dẫn gì".

Đã "Không xem video", thì cần "đọc giáo án".

Có như vậy mới biết khóa học nó làm cái gì.

2. Dành cho học viên "thông thường"

Nguyên tắc:

- Xem video lần lượt
- Xem video kết hợp với giáo án. Bạn không cần take note, vì những điều quan trọng đã có trong giáo án

- Bạn vui lòng code theo video.

Nếu bạn "code theo ý bạn", vui lòng "không hỏi khi có bugs".

Câu chuyện này giống như việc bạn đi khám bệnh, nhưng không tin lời bác sĩ

=> Nếu bạn giỏi, bạn làm luôn bác sĩ, còn đi khám bệnh làm gì.

- Bạn có thể "code theo ý bạn muốn", sau khi "đã kết thúc khóa học"
- Nếu bạn có thắc mắc (hoặc có ý tưởng/nhận thấy bugs), take note lại, bạn hỏi, rồi mình giải đáp.

Chứ không phải là "tự ý làm theo điều các bạn muốn".

Vì đa phần, các bugs trong khóa học mình đã fix hết rồi.

Nên là yên tâm để học theo bạn nhé.

3. Về cách code

Bạn vui lòng code theo video, từ cách đặt tên biến, hàm. Vì mình đã tuân theo "convention tối thiểu" khi bạn đi làm đấy

4. Về bài tập thực hành

Đối với bài tập thực hành, bạn cứ code theo cách bạn hiểu, và kết hợp với "search Google, stackoverflow..."

Ưu tiên cách học thông qua các trang tài liệu chính thức (documents), search Google và đọc stackoverflow.

(đây là cách làm mình hướng dẫn trong khóa học)

Trường hợp đã thử mọi cách không có kết quả, đây là lúc sử dụng các công cụ AI:

- Hạn chế việc, copy/paste solution (tức là hỏi, lấy đáp án, và không hiểu logic thực hiện) từ các ứng dụng AI, ví dụ như ChatGPT
- Với beginners, cần phải xây nền móng thật vững trước khi sử dụng các ứng dụng AI. Nên nhớ, copy/paste nhưng phải hiểu bạn đang làm gì.
- **Nếu bạn sử dụng AI, nên thông qua việc hỏi - phản biện**, giống như việc bạn nhờ AI để thực hiện search/deep-research.
- Nên tự đặt câu hỏi là làm sao có thể code được như vậy, cần học những kiến thức gì, và quan trọng nhất, đừng quên bước thực hành.

Vì không có thực hành, tất cả chỉ là lý thuyết.

#2 + #3 + #4 : Hướng Dẫn Truy Cập Tài Liệu Khóa Học

//todo

@hoidanit

#5. Demo kết quả đạt được

Mục tiêu của khóa học: xây dựng đam mê lập trình với ngôn ngữ Java với người bắt đầu từ số 0 (beginners)

1. Nội dung trọng tâm

Khóa học sẽ chia thành 2 level : basic và advance

Level basic (9 chapters): cú pháp cơ bản, cách khai báo hàm, biến... với Java

Học và áp dụng tư duy lập trình hướng đối tượng (tính đóng gói, kế thừa, đa hình, trừu tượng..)

Level advance (5 chapters): mở rộng với Exception, Generic, Collection Framework...

2. Điểm khác biệt của khóa học

- **Tài liệu chi tiết** theo từng bài giảng, hạn chế việc cần take note
- **Học đi đôi với thực hành** (10 Labs để tự thực hành + rất nhiều bài tập khác theo lý bài học lý thuyết)

3. Đối tượng có thể học

- Học viên chưa biết gì về Java
- Học viên không có đam mê lập trình, học chưa từng học lập trình, muốn đi tìm cảm giác học và hiểu ngôn ngữ lập trình Java
- Học viên có nhu cầu muốn học và nắm vững Java Core

#6. Về Quyền Tác Giả

Bản quyền bài giảng/khóa học (tác phẩm) thuộc về tác giả Hỏi Dân IT

Nghiêm cấm sao chép bài giảng dưới mọi hình thức khi chưa được sự cho phép của tác giả Hỏi Dân IT.

Mọi hành vi cố ý hoặc vô ý vi phạm, đều phải chịu trách nhiệm trước pháp luật.

Vì quyền lợi chính đáng của người học, Hỏi Dân IT (hoidanit.vn) rất mong nhận được sự hợp tác của tất cả các bạn.

Thông báo vi phạm, xin gửi về hòm thư: ads.hoidanit@gmail.com

hoặc inbox trực tiếp Facebook: <https://www.facebook.com/askITwithERIC/>

#7 + #8. Thông Tin Tác Giả Hỏi Dân IT

Về tác giả:

Mọi thông tin về Tác giả Hỏi Dân IT, các bạn có thể tìm kiếm tại đây:

Website chính thức: <https://hoidanit.vn/>

Youtube “Hỏi Dân IT” : <https://www.youtube.com/@hoidanit>

Tiktok “Hỏi Dân IT” : <https://www.tiktok.com/@hoidanit>

Fanpage “Hỏi Dân IT” : <https://www.facebook.com/askITwithERIC/>

Udemy Hỏi Dân IT: <https://www.udemy.com/user/eric-7039/>

Nếu bạn muốn nói chuyện với mình (giao lưu trao đổi võ công :), có thể xem mình livestream trực tiếp tối thứ 2 hàng tuần trên [Youtube Hỏi Dân IT](#)

Chapter 2: Giới thiệu về Java

Cài đặt & chuẩn bị môi trường thực hiện dự án

#9. Tổng quan về các kiến thức sẽ học

Các kiến thức sẽ được đề cập, đi từ dễ tới khó, và bổ trợ kiến thức cho nhau (chương trước là nền tảng để học chương sau)

Phần 1: Lập trình Java Cơ Bản (Java Fundamentals)

Bao gồm:

- **Chapter 2:** Giới thiệu về Java
- **Chapter 3:** Cú pháp cơ bản và biến
- **Chapter 4:** Cấu trúc điều kiện
- **Chapter 5:** Mảng và chuỗi

Phần 2: Lập trình hướng đối tượng (Object Oriented Programming)

Bao gồm:

- **Chapter 6:** Lập trình hướng đối tượng cơ bản
- **Chapter 7:** Tính đóng gói và kế thừa
- **Chapter 8:** Đa hình và trừu tượng
- **Chapter 9:** Enum và Nested Class

Phần 3: Kỹ thuật nâng cao

- **Chapter 10:** Xử lý ngoại lệ
- **Chapter 11:** Collection Framework
- **Chapter 12:** Generics
- **Chapter 13:** Java I/O (input/output)
- **Chapter 14:** Đa luồng trong Java

#10. Java là gì ?

Mục tiêu của video

Trả lời các câu hỏi thường gặp sau:

- Hiểu rõ Java là gì và vai trò của Java trong ngành CNTT.
- Các lĩnh vực ứng dụng của Java.
- Giải đáp thắc mắc "Học Java có lỗi thời không?" bằng góc nhìn thị trường và kỹ thuật.
- Lịch sử phát triển và các đặc điểm nổi bật của Java như: tính độc lập nền tảng, độ bền vững (robustness), bảo mật (security).

1. Java là gì ?

- Java là một ngôn ngữ lập trình hướng đối tượng (OOP) được phát triển bởi Sun Microsystems năm 1995 (sau này được **Oracle mua lại**). (Liệu java có die ?)
- Java có cú pháp tương tự như C++ nhưng đơn giản hơn và loại bỏ nhiều tính năng phức tạp như con trỏ (pointers) (dùng toán tử **this**)
- Java nổi tiếng với slogan: "Write Once, Run Anywhere" (Viết một lần, chạy mọi nơi).
- Tên gọi của Java và logo của java

Code minh họa:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, world!");  
    }  
}
```

2. Ứng dụng của Java

- Ứng dụng **web**: Java EE (Servlet, JSP, Spring)
- Ứng dụng **di động (mobile)**: Android sử dụng Java/Kotlin
- Ứng dụng **desktop**: Swing, JavaFX
- Ứng dụng **doanh nghiệp**: hệ thống ngân hàng, tài chính, bảo hiểm
- Big Data & Cloud: Java được dùng trong Hadoop, Spark
- IoT & nhúng: Java ME cho thiết bị nhúng

Việc làm:

- Java Developer
- Android Developer
- Backend Developer (Spring Boot)
- DevOps/Automation (nhiều tool viết bằng Java: Jenkins, Maven...)

Học Java có lỗi thời không ?

- Không lỗi thời: Java vẫn nằm trong Top 10 ngôn ngữ lập trình phổ biến trên các bảng xếp hạng ([TIOBE](#), [StackOverflow](#)).
- Hệ sinh thái lớn và ổn định: Spring Framework, Java EE, Hibernate,...
- Doanh nghiệp lớn vẫn dùng: Các công ty tài chính, bảo hiểm, ngân hàng không dễ dàng thay đổi sang công nghệ mới.
- Android vẫn hỗ trợ Java (dù Google ưu tiên Kotlin, Java vẫn được dùng rất nhiều).

3. Các đặc điểm nổi bật của Java

- **Platform Independence** (Tính độc lập nền tảng)
Nhờ JVM, code Java biên dịch thành bytecode và có thể chạy trên bất kỳ hệ điều hành nào.
- **Robustness** (Tính bền vững)
Kiểm tra lỗi nghiêm ngặt lúc biên dịch.
Cơ chế quản lý bộ nhớ tự động (Garbage Collector).
- **Security** (Bảo mật)
Java có sandbox để chạy các chương trình đáng ngờ một cách an toàn.
API bảo mật mạnh mẽ: mã hóa, xác thực, quyền truy cập.
- **Object-Oriented**
Mọi thứ trong Java là đối tượng, tăng khả năng tái sử dụng và mở rộng.
- **Multithreaded & Performance**
Hỗ trợ đa luồng (multi-threading) tích hợp sẵn.
Tối ưu hóa hiệu năng qua JIT (Just-In-Time Compiler).

#11. Phân biệt JVM, JRE, JDK

1. Cách đặt tên version của Java

Tên gọi không chính thức	Tên gọi chuẩn	Ghi chú
Java 1.0 – 1.4	JDK 1.0 – 1.4	Các phiên bản đầu tiên, dùng số 1.x
Java 5	JDK 5	Bắt đầu bỏ số 1, dùng trực tiếp số phiên bản
Java 6 → Java 8 Java 8 (18/03/2014)	JDK 6 → JDK 8	Thường dùng cả hai cách viết
Java 9 → Java 21+	JDK 9 → JDK 21	Chu kỳ phát hành 6 tháng/lần từ Java 9

[Version mới nhất](#) tại thời điểm quay video này là Java 24 (release ngày 18/03/2025)

2. Phân biệt JVM, JRE, JDK

JVM – Java Virtual Machine: Là máy ảo Java giúp chạy các file .class (bytecode).

Quá trình hoạt động:

- Code Java → biên dịch bằng **javac** → tạo file **.class**
- **JVM đọc bytecode từ file .class** và thực thi trên máy thật
- JVM giúp Java có tính "Write Once, Run Anywhere"
Ví dụ: Bạn viết code trên Windows, biên dịch ra bytecode, có thể chạy trên Linux hoặc Mac nhờ JVM.

JRE – Java Runtime Environment: Là môi trường để chạy các ứng dụng Java.

Bao gồm:

- **JVM: Java Virtual Machine**
- Thư viện lớp chuẩn (Standard Class Libraries)
- JRE không bao gồm trình biên dịch – chỉ để chạy, không thể lập trình được.
- Người dùng bình thường cài JRE là đủ để chạy các phần mềm viết bằng Java.

JDK – Java Development Kit: Là bộ công cụ đầy đủ để lập trình Java.

Bao gồm:

- JRE (tức là JVM + thư viện chuẩn)
- Trình biên dịch (javac)
- Công cụ dòng lệnh: java, jar, javadoc, jshell...
- **Dành cho lập trình viên – bắt buộc phải cài nếu muốn lập trình Java.**

Thành phần	Bao gồm	Mục đích sử dụng
JVM	Bytecode	Thực thi chương trình Java
JRE	JVM + thư viện	Chạy ứng dụng Java
JDK	JRE + công cụ dev	Viết, biên dịch và chạy Java

#12. Chuyện Cài Đặt Công Cụ (Bắt Buộc Xem)

1. Mục đích

Chương học này sẽ hướng dẫn chi tiết cách cài đặt các công cụ cần thiết để phục vụ cho khóa học. Vì vậy, **bạn vui lòng không bỏ qua video nào, xem lần lượt theo thứ tự**

Có 2 sai lầm mà các bạn hay gặp phải, đặc biệt là những bạn “đã biết 1 chút”

Sai lầm 1: Bỏ qua các video cài đặt công cụ vì bạn cho rằng bạn “đã biết rồi”

Hãy nhớ rằng, **cài công cụ là 1 phần, đang còn phải “cấu hình” nó nữa.**

Khóa học được sinh ra, và đã tối ưu. Bạn chỉ dành 5 tới 10 phút để xem video, đổi lại tiết kiệm cho bạn cả giờ đồng hồ ngồi mò mẫm.

Sai lầm 2: Không quan tâm tới version của phần mềm

Khi thực hiện khóa học, **bạn vui lòng download và cài đặt version phần mềm giống như video. Điều này sẽ đảm bảo môi trường thực thi code là giống nhau. (hạn chế tối đa bug có thể xảy ra)**

2. Version Phần Mềm theo thời gian

Bạn đừng sợ phần mềm (công cụ) nó thay đổi version, hay thậm chí là “chê bai” version cũ. Vì vốn dĩ, công nghệ nó là vậy, luôn thay đổi theo thời gian.

Bạn yêu cầu “version mới nhất” cho cái bạn học, mình đã làm điều đó tại thời điểm quay video khóa học, tuy nhiên, sẽ là cố định 1 version.

Lý do: công nghệ sẽ cập nhật theo thời gian. Cho dù bạn muốn hay không, hoặc thậm chí lấp tên lửa vào đ*t, đuổi cũng không kịp.

Những cái ngày hôm nay, bạn cho là mới nhất, qua ngày mai, nó đã là cũ.

Điều bạn cần làm là: học 1 version, và quan trọng hơn, là bạn học, bạn cần hiểu nó

Sau đấy, nếu cần thiết, bạn học version mới hơn. Điểm khác biệt ở đây, là khi học version mới, bạn không phải là người bắt đầu từ số 0 (do đã có base từ version cũ)

Chỉ không học version cũ khi và chỉ khi: sản phẩm của version cũ không dùng được

Đây là lý do tại sao các khóa học của mình, khi học xong, mình mới hướng dẫn nâng cấp version.

#13. Cài Đặt Môi Trường Java

1. Version cài đặt

- Nên chọn version LTS (long term support)

<https://www.oracle.com/java/technologies/java-se-support-roadmap.html>

- Khóa học này sử dụng Java version 21. (hoặc 17)

Vì vậy, để hạn chế lỗi tối đa có thể xảy ra, mình khuyến khích sử dụng version trên

2. Cài đặt Java

Download Java [tại đây](#)

[Thiết lập](#) biến môi trường **JAVA_HOME**

#14. Cài Đặt Công Cụ Coding Java

Editor/IDE (Integrated Development Environment) cần đáp ứng các tiêu chí:

- Giá cả hợp lý (ưu tiên mã nguồn mở - miễn phí)
- Dễ sử dụng
- Hỗ trợ cú pháp coding (gợi ý code, phát hiện lỗi)
- Hỗ trợ debug
- Cộng đồng sử dụng rộng rãi (community support)
- Hỗ trợ đa nền tảng: Windows, MacOS, Linux

1. Các công cụ phổ biến để code Java:

Dùng miễn phí:

- [Eclipse](#)
- [VSCode](#): cần cài thêm extension để code Java
- [IntelliJ IDEA Community](#) : hỗ trợ Java
- Spring Tool Suite (STS): chuyên dùng cho Java Spring (phát triển từ Eclipse)

Dùng trả phí: [IntelliJ IDEA Ultimate](#)

Trong khóa học này, mình sử dụng Spring Tool Suite (và VSCode, nếu có), vì miễn phí, hỗ trợ tốt cho việc code Java

2. Cài đặt Spring Tool Suite (STS)

Tải version mới nhất [tại đây](#)

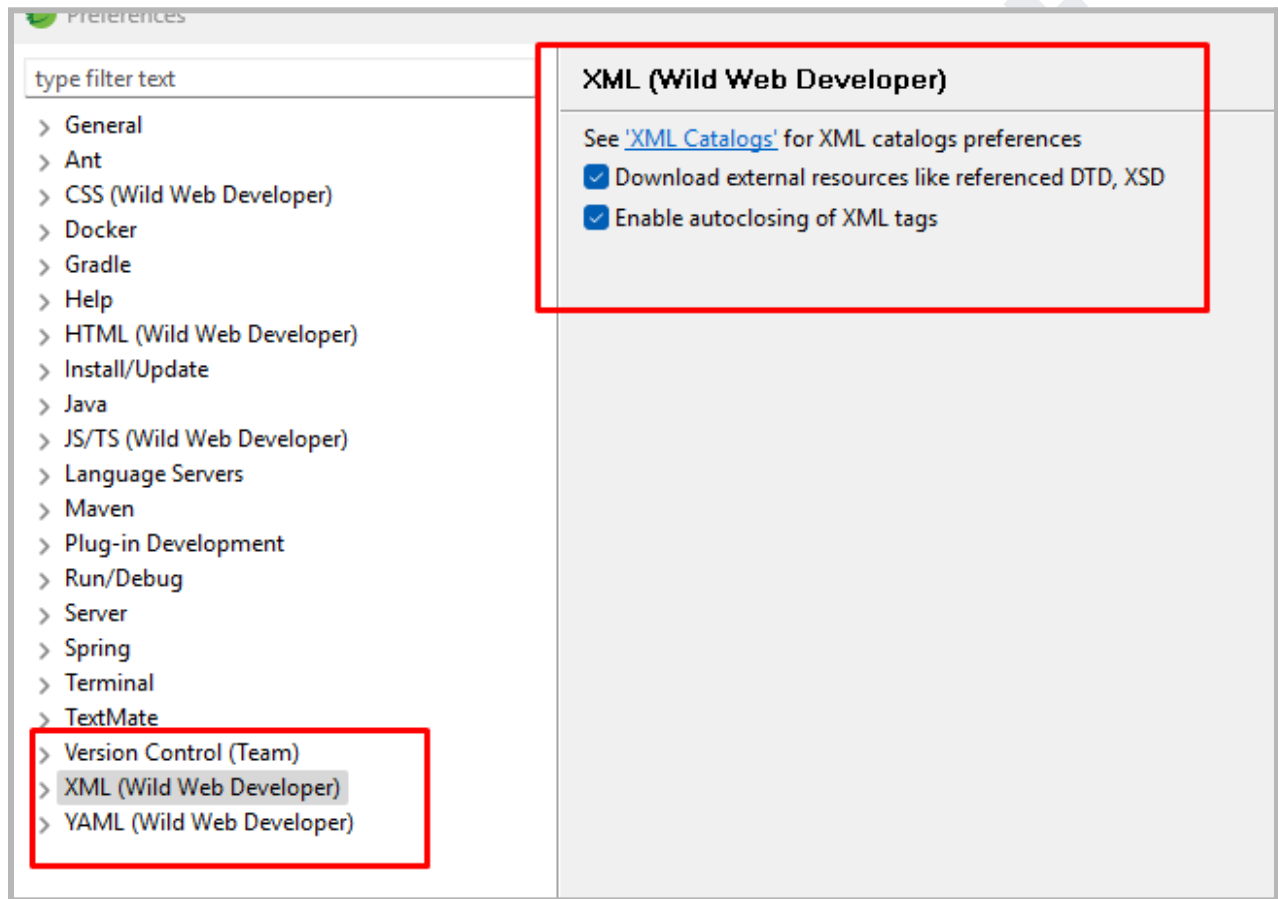
Tải tất cả version [tại đây](#)

3. Cấu Hình Spring Tool Suite

//cấu hình download pom.xml

Open the Eclipse preferences window (**Window > Preferences**), Go to **XML**(Wild Web Developer).

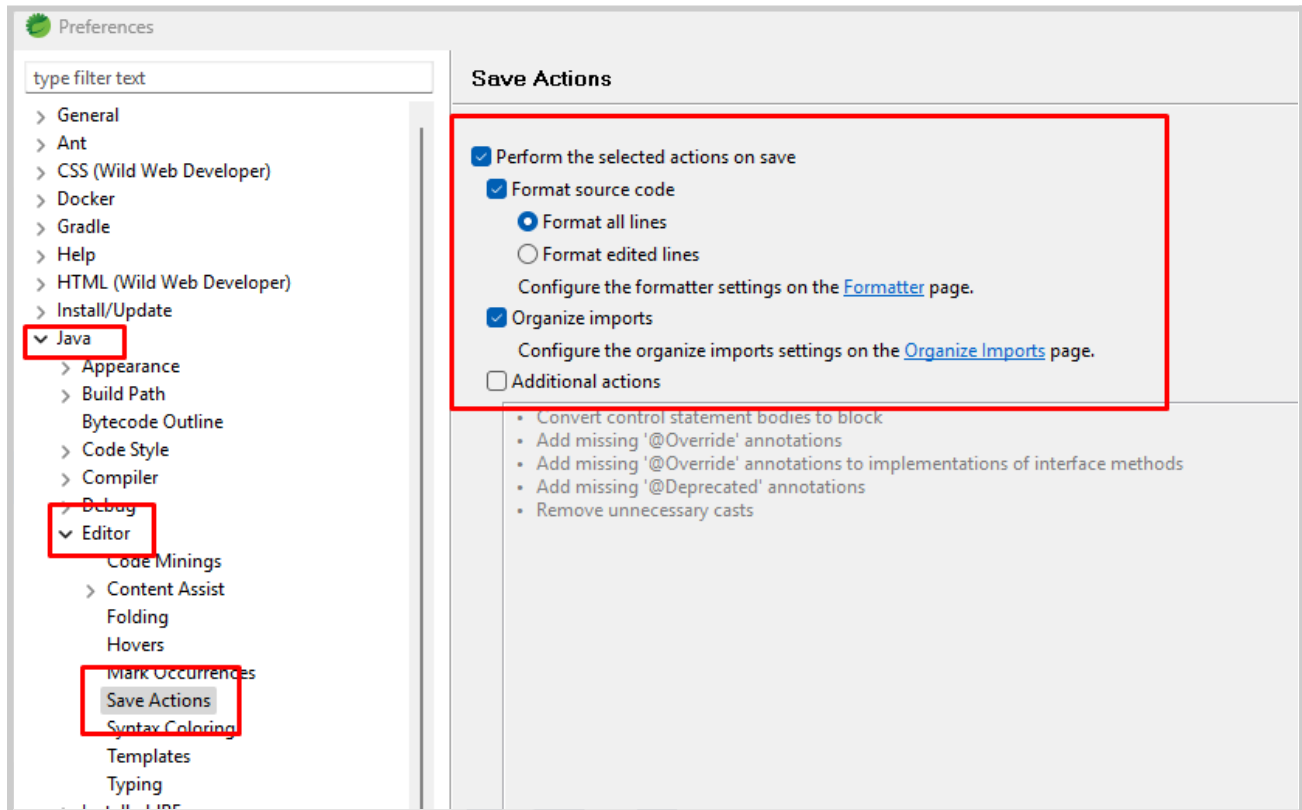
check the "Download external resources like referenced DTD,XSD", then Apply.



//cấu hình format on save

Go to Window > Preferences (or Eclipse > Preferences on macOS).

Expand Java > Editor > Save Actions.



#15. Hello World với Java

Bước 1: Tạo folder chứa code

Lưu ý về tên dự án thực hành và đường dẫn (path) nơi lưu trữ code, không dùng ký tự đặc biệt và tiếng việt có dấu

Nên dùng: D://my-project/java-hoidanit

Không nên dùng: D://dự án/học java

Đặt tên dự án thực hành là: java-core-hoidanit

Bước 2: Tạo file java

- Tạo file **HelloWorld.java**

//nội dung như sau

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World @hoidanit!");  
    }  
}
```

Bước 3: Chạy dự án thực hành

//todo

#16. Quá trình thực thi Java (Extra)

Bước 1: Mở Command Prompt (CMD)

Di chuyển đến thư mục chứa file HelloWorld.java

Bước 2: Biên dịch bằng javac

javac HelloWorld.java

Sau lệnh này, một file mới **HelloWorld.class** sẽ được tạo ra – đây là bytecode.

Bước 3: Chạy chương trình bằng java (chạy file .class)

java HelloWorld

Quy trình:

[HelloWorld.java] --> (javac) --> [HelloWorld.class] --> (java) --> Hello, World!

#17. Sử dụng Git để quản lý mã nguồn

Mục đích: công cụ để quản lý mã nguồn và sử dụng source code của dự án (tránh tình trạng máy tính hư bị mất hết code)

Nếu bạn chưa biết gì về Git, xem nhanh [tại đây](#) (miễn phí)

Khóa học Git trả phí, tham khảo [tại đây](#)

- Sử dụng Git theo nguyên tắc:

1. Học xong video nào, commit đẩy lên Github/Gitlab

=> tạo cơ hội để thực hành câu lệnh của Git, ví dụ:

git add

git commit

git push...

2. Git là công cụ "mặc định bạn phải biết" khi đi làm phần mềm

=> điều 1 ở trên giúp bạn thực hành

3. Thói quen học xong video nào, đẩy code lên Git, giúp bạn tạo ra bản "backup" cho project của bạn

Ví dụ máy tính bạn bị hỏng đột xuất/bị mất

=> vẫn còn code, chỉ cần pull về code tiếp mà không phải code từ đầu.

=> Mục đích sử dụng git ở đây là : backup code + thực hành công cụ đi làm mà bạn "phải biết" nếu muốn đi thực tập/đi làm.

Chapter 3: Cú pháp cơ bản và biến

Tìm hiểu và nắm vững cách khai báo cú pháp cơ bản và việc sử dụng biến số của Java

#18. Tổng quan về chapter

//todo

#19. Variables (Khái niệm biến)

Biến số (cách khai báo biến số) là nền tảng bắt buộc trước khi học logic, toán tử hay điều kiện trong Java.

1. Biến (variable) trong Java là gì ?

Biến là **vùng nhớ** được đặt tên để lưu dữ liệu **tạm thời** trong chương trình.

(Biến = tên đại diện cho vùng nhớ RAM, dùng để lưu tạm dữ liệu trong lúc chương trình đang chạy)

Cú pháp:

<kiểu_dữ_liệu> <tên_biến> = <giá_trị>;

Ví dụ: `int age = 25;`

Lưu ý: với java, cần có dấu chấm phẩy (;) sau khi đã khai báo

2. Quy tắc đặt tên biến

- Sử dụng ký tự alphabet (a-zA-Z), dấu \$ hoặc dấu gạch dưới(_)
- Tên biến có phân biệt HOA/thường
- Không đặt tên bắt đầu bởi số, và không dùng các từ khóa được sử dụng để xây dựng ngôn ngữ java, ví dụ: new, class, const...

Ví dụ:

Tên biến nào sau đây không hợp lệ:

abc

1abc

ab_c

_abc

\$abc

ab-c

_123

\$123

if

\$if

#20. Kiểu Dữ Liệu Nguyên Thủy (Primitive Types)

1. Dữ liệu nguyên thủy (8 loại) (Primitive Types)

Kiểu dữ liệu	Kích thước	Phạm vi giá trị	Ví dụ
byte	8-bit	-128 đến 127	byte b = 10;
short	16-bit	-32,768 đến 32,767	short s = 1000;
int	32-bit	-2^{31} đến $2^{31}-1$	int a = 100000;
long	64-bit	-2^{63} đến $2^{63}-1$	long l = 100000000000L;
float	32-bit	$\sim -3.4 \times 10^{38}$ to 3.4×10^{38} Độ chính xác (precision): ~6–7 chữ số thập phân	float f = 3.14f;
double	64-bit	$\sim -1.7 \times 10^{308}$ to 1.7×10^{308} Độ chính xác (precision): ~15–16 chữ số thập phân	double d = 3.141592d;
char	16-bit	0 đến 65,535 (ký tự Unicode)	char c = 'A';
boolean	1 bit	true / false	boolean b = true;

2. Ví dụ

```
public class PrimitiveExample {  
    public static void main(String[] args) {  
        int age = 20;  
        float height = 1.75f;  
        char grade = 'A';  
        boolean isPassed = true;  
  
        System.out.println("Age: " + age);  
        System.out.println("Height: " + height);  
        System.out.println("Grade: " + grade);  
        System.out.println("Passed? " + isPassed);  
    }  
}
```

#21. Kiểu Dữ Liệu Tham Chiếu (Reference Types)

1. Dữ liệu tham chiếu

Kiểu	Mô tả	Ví dụ
String	Chuỗi ký tự	String name = "hoidanit";
Object	Lớp cha của mọi đối tượng	Object obj = new Object();
Array	Mảng các phần tử	int[] numbers = {1, 2, 3};
List, Map, Set	Các cấu trúc dữ liệu nâng cao (sẽ học sau)	

2. Ví dụ

```
public class ReferenceExample {  
    public static void main(String[] args) {  
        String name = "hoidanit";  
        String city = "Hà Nội";  
        int[] scores = {90, 85, 78};  
        System.out.println("Tên: " + name);  
        System.out.println("Thành phố: " + city);  
        System.out.println("Điểm: " + scores[0] + ", " + scores[1] + ", " + scores[2]);  
    }  
}
```

3. Tại sao lại phân chia thành kiểu dữ liệu nguyên thủy và tham chiếu

Lý do	Chi tiết
Hiệu suất	Dữ liệu đơn giản không cần xử lý như đối tượng, giúp chương trình chạy nhanh hơn
Thiết kế rõ ràng	Tách biệt giữa dữ liệu cơ bản và cấu trúc phức tạp giúp code dễ đọc, dễ hiểu
Tối ưu hóa JVM	JVM có thể áp dụng các kỹ thuật tối ưu riêng cho primitive và reference types
Hỗ trợ OOP	Cần kiểu tham chiếu để thao tác với các đối tượng trong Java.

#22. Boxing và Unboxing

1. Wrapper Classes – Lớp bao (đóng gói)

Java cung cấp một số lớp để “bao” các kiểu dữ liệu nguyên thủy thành đối tượng.

Kiểu nguyên thủy	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Vì sao cần wrapper class?

- Dùng với collections như ArrayList, HashMap (chỉ chấp nhận object).
- Cung cấp thêm các hàm tiện ích, ví dụ: Integer.parseInt("123")

//todo: lấy ví dụ minh họa

2. Boxing (Autoboxing) – Chuyển từ primitive → object

Java tự động chuyển kiểu nguyên thủy thành đối tượng wrapper khi cần.

```
int x = 10;
```

```
Integer y = x; // autoboxing: int → Integer
```

```
=== Integer y = Integer.valueOf(x);
```

3. Unboxing – Chuyển từ object → primitive

Java cũng tự động chuyển từ wrapper class về kiểu nguyên thủy.

```
Integer a = 15;
```

```
int b = a; // unboxing: Integer → int
```

```
===
```

```
int b = a.intValue();
```

#23. Toán Tử Trong Java

1. Toán Tử Số Học

Toán tử	Miêu tả	Ví dụ (a = 10, b = 3)
+	Cộng	$a + b = 13$
-	Trừ	$a - b = 7$
*	Nhân	$a * b = 30$
/	Chia	$a / b = 3$ (nguyên)
%	Chia lấy dư	$a \% b = 1$

2. Toán Tử So Sánh

Toán tử	Miêu tả	Ví dụ (a = 10, b = 3)
==	Bằng	$a == b \rightarrow \text{false}$
!=	Khác	$a != b \rightarrow \text{true}$
>	Lớn hơn	$a > b \rightarrow \text{true}$
<	Nhỏ hơn	$a < b \rightarrow \text{false}$
>=	Lớn hơn hoặc bằng	$a >= 10 \rightarrow \text{true}$
<=	Nhỏ hơn hoặc bằng	$a <= 5 \rightarrow \text{false}$

3. Toán Tử Logic

Toán tử	Miêu tả	Ví dụ (a = 10, b = 3)
&&	Và (AND)	(a > 5 && b < 5) → true
!	Phủ định (NOT)	!(a > 5) → false

#24. Nhập và Xuất Dữ Liệu Trong Java

//todo: giải thích code - hàm main

1. Xuất Dữ Liệu – System.out.print và System.out.println

System.out.print(...): In không xuống dòng.

Ví dụ:

```
System.out.print("Hello ");  
System.out.print("World"); // → Hello World
```

System.out.println(...): In có xuống dòng. (line)

Ví dụ:

```
System.out.println("Hello");  
System.out.println("World"); // → Hello\nWorld
```

Kết hợp in chuỗi + biến:

Ví dụ:

```
int age = 20;  
System.out.println("Tuổi của bạn là: " + age);
```

System.out.printf(); //format

%d: số nguyên

%f: số thực

- mặc định là 6 số lẻ

%.3f định dạng 3 số lẻ

%s: chuỗi

Ví dụ:

```
System.out.print("Hỏi Dân IT");
```

```
System.out.println("với Eric");
```

```
System.out.printf("có %d người đăng ký", 70000);
```

2.Nhập Dữ Liệu – Dùng lớp Scanner

Bước 1: Khởi tạo Scanner

```
import java.util.Scanner;
```

```
Scanner scanner = new Scanner(System.in);
```

```
//Scanner đọc dữ liệu từ System.in (bàn phím)
```

Bước 2: Các phương thức nhập phổ biến của Scanner

Phương thức	Kiểu dữ liệu	Ví dụ
nextLine()	Chuỗi (String)	String name = scanner.nextLine();
nextInt()	Số nguyên	int age = scanner.nextInt();
nextDouble()	Số thực	double height = scanner.nextDouble();

Bước 3: Lỗi thường gặp khi dùng Scanner

```
int age = scanner.nextInt();
```

```
String name = scanner.nextLine(); // bị bỏ qua!
```

Fix:

```
int age = scanner.nextInt();
```

```
scanner.nextLine(); // Đọc bỏ dòng thừa
```

```
String name = scanner.nextLine(); // đọc chính xác
```

#25. Bài Tập Lab 01

Mục tiêu:

- Biết cách tạo 1 java project đơn giản với Spring Tool Suite
- Đọc dữ liệu từ bàn phím
- In giá trị ra console

Bài 1:

Viết chương trình cho phép nhập tên sinh viên, điểm trung bình từ bàn phím.

In ra màn hình với định dạng: <tên_sinh_viên> có điểm = <điểm>

Ví dụ: "Eric" có điểm = 9.6

Trong đó: <tên sinh viên>: Eric

<điểm> : 9.6

Gợi ý:

- Chạy spring tool suite để tạo dự án java
- tạo class Lab1Bai1
- tạo hàm main
- Đọc kết quả từ bàn phím:

```
Scanner scanner = new Scanner(System.in);
```

```
String name = scanner.nextLine();
```

- In kết quả với println:

```
System.out.println("Your name is: " + name);
```

Bài 2: Viết chương trình nhập từ bàn phím 2 cạnh của 1 hình chữ nhật.

Tính toán và in ra console:

- chu vi hình chữ nhật
- diện tích hình chữ nhật
- cạnh nhỏ nhất của hình chữ nhật

Gợi ý:

chu vi = (dài + rộng) x 2

diện tích = dài x rộng

cạnh nhỏ nhất = Math.min(dài, rộng)

Bài 3: Viết chương trình nhập vào bàn phím cạnh của một khối lập phương.

Tính và xuất ra thể tích của hình lập phương.

Gợi ý:

thể tích = cạnh x cạnh x cạnh = cạnh^3 = Math.pow(cạnh, 3)

#26. Chữa Bài Tập Lab 01

//todo

Chapter 4: Cấu trúc điều khiển

Tìm hiểu cách sử dụng câu điều kiện với Java

#27. Tổng quan về chapter

//todo

#28. Câu lệnh điều kiện - if, if-else, if-else if

Mục tiêu:

- Hiểu cách hoạt động của cấu trúc rẽ nhánh.
- Biết sử dụng **if**, **if-else**, **if-else if** để xử lý các điều kiện.

Cú pháp:

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

Ví dụ:

```
int score = 75;  
if (score >= 90) {  
    System.out.println("Excellent");  
} else if (score >= 70) {  
    System.out.println("Good");  
} else {  
    System.out.println("Needs Improvement");  
}
```

#29. Bài Tập Câu Lệnh Điều Kiện

Yêu cầu: Viết chương trình Java nhập vào điểm trung bình của một học sinh, sau đó in ra xếp loại học lực theo thang điểm 10 như sau:

Điểm	Học lực
9.0 – 10.0	Xuất sắc
8.0 – 9.0	Giỏi
6.5 – 8.0	Khá
5.0 – 6.5	Trung bình
0 – 5	Yếu
Ngoài khoảng	Không hợp lệ

#30. Câu lệnh switch-case

Mục tiêu:

Biết cách sử dụng switch để thay thế if-else nhiều điều kiện.

Cú pháp:

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

Ví dụ:

```
int day = 3;  
switch (day) {  
    case 1: System.out.println("Monday"); break;  
    case 2: System.out.println("Tuesday"); break;  
    default: System.out.println("Another day");  
}
```

Với java 12 trở đi (giới thiệu mở rộng)

Từ Java 12: switch expression dùng -> và cho phép gán giá trị trực tiếp.

<https://blogs.oracle.com/javamagazine/post/new-switch-expressions-in-java-12>

//nhỏ hơn 12

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Another day");
}
```

//từ version 12 trở đi

```
String result = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Another day";
};
System.out.println(result);
```

#31. Bài Tập Câu Lệnh Switch-Case

Yêu cầu:

Viết chương trình Java, nhập vào một số nguyên từ 1 đến 7.

In ra tên ngày trong tuần tương ứng, theo thứ tự:

1: Monday

2: Tuesday

3: Wednesday

4: Thursday

5: Friday

6: Saturday

7: Sunday

Nếu người dùng nhập ngoài phạm vi 1–7, in ra: "Invalid day number!"

#32. Vòng lặp for, while, do-while

Khi lập trình, có nhiều thao tác cần lặp lại (ví dụ: tính tổng từ 1 đến 100, in danh sách sản phẩm...).

Câu hỏi: Nếu cần in ra số từ 1 đến 100, bạn sẽ viết 100 dòng `System.out.println()`?

Sử dụng vòng lặp – giúp lặp lại đoạn mã một cách gọn gàng, linh hoạt.

1. Vòng lặp for

Được sử dụng khi biết trước số lần lặp

Cú pháp:

```
for (int i = 0; i < n; i++) {  
    // code  
}
```

Ví dụ:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("Lần " + i);  
}
```

2. Vòng lặp while

Không biết trước số lần lặp, kiểm tra điều kiện trước.

Cú pháp:

```
while (điều_kiện) {  
    // code  
}
```

Ví dụ:

```
int i = 1;  
while (i <= 5) {  
    System.out.println("While lần " + i);  
    i++;  
}
```

3. Vòng lặp do-while

Luôn thực hiện ít nhất 1 lần.

Cú pháp:

```
do {  
    // code  
} while (điều_kiện);
```

Ví dụ:

```
int i = 1;  
do {  
    System.out.println("Do-While lần " + i);  
    i++;  
} while (i <= 5);
```

#33. Bài Tập Sử Dụng Vòng Lặp (Loop)

Yêu cầu 1: Tính tổng từ 1 đến 100 (dùng for)

Yêu cầu 2: In số chẵn từ 2 đến 20 (dùng while)

Yêu cầu 3:

Viết một chương trình Java yêu cầu người dùng nhập mật khẩu từ bàn phím.

Nếu người dùng nhập sai mật khẩu, chương trình sẽ yêu cầu nhập lại.

Khi người dùng nhập đúng mật khẩu (ví dụ: "hoidanit"), chương trình sẽ in ra thông báo "Đăng nhập thành công!" và kết thúc vòng lặp.

Sử dụng vòng lặp **do-while** để đảm bảo rằng chương trình luôn yêu cầu nhập ít nhất một lần.

Lưu ý: So sánh chuỗi dùng `.equals()` thay vì `==`

#34. Câu lệnh điều khiển luồng – break, continue, return

1. Câu lệnh break

Mục đích: Thoát khỏi vòng lặp hiện tại ngay lập tức.

Cú pháp:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    System.out.println(i);  
}  
// Output: 1 2 4 5
```

2. Câu lệnh continue

Mục đích: Bỏ qua lượt lặp hiện tại và chuyển sang lượt tiếp theo.

Cú pháp:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    System.out.println(i);  
}  
// Output: 1 2 4 5
```

3. Giới thiệu về câu lệnh return

```
//todo
```

#35. Bài Tập Câu Lệnh Điều Khiển Luồng (Break/Continue)

Yêu cầu 1: Tìm số đầu tiên chia hết cho 7 trong dãy số từ 1 đến 100

Gợi ý: Duyệt từ 1 → 100, in ra số đầu tiên chia hết cho 7 rồi dừng lại (dùng break).

Yêu cầu 2: In các số từ 1 đến 10, bỏ qua số lẻ (dùng continue)

Gợi ý: Nếu $i \% 2 \neq 0$ thì continue.

#36. Bài Tập Lab 02

Mục tiêu: Vận dụng linh hoạt các kiến thức đã học trong chương

- Câu lệnh if, if-else, if-else if
- switch-case
- Vòng lặp for, while, do-while
- Câu lệnh điều khiển break, continue

Yêu cầu:

Viết một chương trình Java (Quản lý điểm số), giúp người dùng nhập và xử lý một số điểm, với các chức năng xử lý sau:

Tính năng 1: Nhập điểm số (dùng do-while)

- Cho phép người dùng liên tục nhập điểm từ bàn phím (giá trị 0 đến 10).
- Nếu nhập điểm ngoài khoảng, yêu cầu nhập lại (dùng continue).
- Người dùng có thể dừng nhập khi gõ -1
- Lưu các điểm đã nhập vào biến tổng và đếm số lượng để tính trung bình.

Ví dụ về luồng chạy:

Nhập điểm (từ 0 đến 10, nhập -1 để dừng): 9

Nhập điểm: 5

Nhập điểm: -3

Điểm không hợp lệ. Nhập lại.

Nhập điểm: 11

Điểm không hợp lệ. Nhập lại.

Nhập điểm: -1

Tính năng 2: Hiển thị Menu lựa chọn (dùng switch-case)

Sau khi nhập xong (tại tính năng 1, nhập -1), hiển thị menu:

===== MENU =====

1. Tính điểm trung bình
2. Phân loại học lực
3. Thoát chương trình

=====

Nhập lựa chọn:

Lựa chọn 1: Tính điểm trung bình (dùng if)

Tính tổng các điểm đã nhập và in ra kết quả trung bình làm tròn 2 chữ số.

Lựa chọn 2: Phân loại học lực (dùng if-else if)

Dựa vào trung bình:

- 9 – 10: Xuất sắc
- 8 – dưới 9: Giỏi
- 6.5 – dưới 8: Khá
- 5 – dưới 6.5: Trung bình
- Dưới 5: Yếu

Lựa chọn 3: Thoát chương trình

Dùng **return** để kết thúc main.

#37. Chữa Bài Tập Lab 02

//todo

Chapter 5: Mảng và Chuỗi

Tìm hiểu cách hoạt động của mảng (array) và chuỗi (String) trong java

#38. Tổng quan về chapter

//todo

#39. Mảng 1 Chiều – Cấu trúc, Khởi tạo, Duyệt mảng (Array)

Mục tiêu:

- Hiểu mảng là gì và tại sao cần dùng.
- Biết cách khai báo, khởi tạo, gán giá trị và truy cập mảng 1 chiều.
- Biết cách dùng vòng lặp for để duyệt mảng.

1.Mảng (Array) là gì ?

- Tập hợp các phần tử **cùng kiểu dữ liệu**, lưu trong bộ nhớ liên tiếp.
- Mỗi phần tử được truy cập qua **chỉ số (index) bắt đầu từ 0**.

Cú pháp khai báo:

```
int[] a = new int[5];    // mặc định là 0
int[] b = {1, 2, 3, 4, 5}; // khởi tạo trực tiếp
```

Truy cập và thao tác với mảng:

```
a[0] = 10;
System.out.println(a[0]); // in giá trị đầu tiên
```

Lỗi thường gặp: `ArrayIndexOutOfBoundsException`

Duyệt mảng với vòng lặp:

```
for (int i = 0; i < b.length; i++) {
    System.out.println(b[i]);
}
```

#40. Bài Tập Mảng 1 Chiều

Yêu cầu: Viết một chương trình Java thực hiện các bước sau:

- Yêu cầu người dùng nhập vào 5 số nguyên từ bàn phím.
- Lưu các số này vào một mảng 1 chiều.
- Tính tổng của tất cả phần tử trong mảng.
- Tìm và in ra số lớn nhất trong mảng.

Ví dụ In kết quả ra màn hình theo mẫu:

Tổng các số là: 52

Số lớn nhất là: 15

#41. Giới Thiệu Mảng Đa Chiều (Extra)

1. Mảng đa chiều là gì

- Mảng đa chiều là **mảng của mảng**.
- Cấu trúc này cho phép **lưu trữ dữ liệu dạng bảng, lưới, ma trận...**
- Mỗi phần tử trong mảng đa chiều lại là một mảng khác.

Mảng 2 chiều là loại phổ biến nhất, còn gọi là ma trận.

Ví dụ:

```
Int[ ][ ] arr = new int[2][3]; // 2 hàng, 3 cột. Mặc định giá trị là 0
```

```
Int[ ][ ] matrix = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Có 2 hàng (row), mỗi hàng có 3 cột (column):

Hàng 0 → [1, 2, 3]

Hàng 1 → [4, 5, 6]

Cách truy cập: tênMảng[hàng][cột]

Ví dụ:

```
System.out.println(arr[0][1]); // In ra phần tử hàng 0, cột 1 → 20
```

Duyệt mảng với vòng lặp:

```
for (int i = 0; i < arr.length; i++) {  
    for (int j = 0; j < arr[i].length; j++) {  
        System.out.print(arr[i][j] + " ");  
    }  
    System.out.println(); //empty line  
}
```

2. Ứng dụng của mảng đa chiều

- Bàn cờ hoặc trò chơi ma trận

Ví dụ cờ caro 3x3:

```
char[ ][ ] board = {  
    {'X', 'O', 'X'},  
    {' ', 'X', 'O'},  
    {'O', ' ', 'X'}  
};
```

Hiển thị trạng thái ván cờ

Kiểm tra người thắng cuộc (3 ký tự giống nhau theo dòng/cột/chéo)

- Hình ảnh kỹ thuật số (ảnh đen trắng hoặc màu)

Ảnh có thể xem là ma trận điểm ảnh (pixels)

Ảnh đen trắng: `int[ ][ ] imageGray`

Ảnh màu: `int[ ][ ][ ] imageColor` (3 chiều – RGB)

#42. Vòng lặp for-each trong Java

1. for-each là gì?

for-each (còn gọi là **enhanced for loop**) là **cách viết ngắn gọn hơn của vòng lặp for** khi bạn cần **duyệt qua toàn bộ phần tử của mảng hoặc danh sách (array, collection)**.

Cú pháp dễ đọc, không cần dùng chỉ số i, giảm sai sót.

Cú pháp:

```
for (KiểuDữLiệu biến : tênMảng) {  
    // Sử dụng biến  
}
```

Ví dụ:

```
int [] numbers = {1, 2, 3, 4, 5};
```

```
for (int num : numbers) {  
    System.out.println(num);  
}
```

2. Ưu điểm của for-each

- Ngắn gọn, dễ đọc: không cần chỉ số i, tránh lỗi ngoài mảng
- Không thể truy cập sai index: tránh lỗi `ArrayIndexOutOfBoundsException`
- An toàn, sạch sẽ: phù hợp với các tập hợp (collections)

3. Nhược điểm của for-each

Không biết được chỉ số (index): không thể biết phần tử đang ở vị trí mấy

#43. Bài Tập for-each

Yêu cầu: Viết chương trình Java thực hiện các bước sau:

Khai báo một mảng double[] chứa điểm của học sinh ở các môn: {8.5, 7.0, 9.0, 6.5, 8.0}

Duyệt mảng bằng vòng lặp **for-each** để:

- Tính tổng điểm
- Tính và in ra điểm trung bình

#44. Lớp String & Các Phương Thức Phổ Biến

Mục tiêu:

Biết cách khai báo, khởi tạo và sử dụng các phương thức thông dụng của String. (kiểu dữ liệu được dùng nhiều nhất)

1.String (chuỗi)

Cách khai báo:

```
String s1 = "Java";
```

```
String s2 = new String("hoidanit");
```

Các phương thức thường dùng:

Phương thức	Tác dụng
length()	Độ dài chuỗi
charAt(int)	Lấy ký tự tại vị trí
substring(start, end)	Cắt chuỗi con
indexOf(String)	Tìm vị trí ký tự/chuỗi
toUpperCase()	Chuyển thành chữ in hoa
trim()	Xóa khoảng trắng đầu/cuối

Ví dụ:

```
String name = " Java Basic ";
```

```
System.out.println(name.trim());    // "Java Basic"
```

```
System.out.println(name.length());  // 13 (bao gồm khoảng trắng)
```

```
System.out.println(name.indexOf("a")); // 2
```

#45. So sánh chuỗi == vs equals()

Mục tiêu:

- Hiểu sự khác biệt giữa == và .equals() khi so sánh chuỗi.
- Tránh lỗi phổ biến khi so sánh chuỗi bằng ==.

1. So sánh địa chỉ (==)

Ví dụ:

```
String a = "hello";  
String b = new String("hello");  
System.out.println(a == b);    // false
```

2. So sánh nội dung (equals())

```
System.out.println(a.equals(b)); // true
```

Kinh nghiệm với Java: So sánh chuỗi → luôn dùng .equals() để so sánh nội dung/giá trị

Ví dụ:

```
String s1 = "abc";  
String s2 = "abc";  
String s3 = new String("abc");  
  
System.out.println(s1 == s2);    // true (do tối ưu bộ nhớ)  
System.out.println(s1 == s3);    // false  
System.out.println(s1.equals(s3)); // true
```

#46. Xử lý chuỗi hiệu quả với StringBuilder & StringBuffer (Extra)

1.String là immutable (Không thay đổi)

Trong Java, immutable nghĩa là không thể thay đổi sau khi được tạo ra.

Khi bạn thay đổi chuỗi String, Java không chỉnh sửa nội dung gốc, mà tạo ra một đối tượng chuỗi mới.

Ví dụ:

```
String s = "Hello";  
s = s + " World";  
System.out.println(s); // Hello World
```

Quá trình xảy ra:

- Ban đầu s tham chiếu đến chuỗi "Hello".
- Khi bạn nối " World" bằng s + " World", Java tạo một đối tượng chuỗi mới chứa "Hello World".
- Biến s không thay đổi nội dung cũ, mà trở sang đối tượng mới.
- Đối tượng "Hello" vẫn tồn tại trong bộ nhớ (chờ GC thu dọn).

Hậu quả khi dùng String trong vòng lặp nối chuỗi:

```
String result = "";  
for (int i = 0; i < 1000; i++) {  
    result = result + i; // mỗi lần + tạo một đối tượng mới  
}
```

Với 1000 vòng lặp → Java tạo ra hàng trăm đối tượng String mới → rất tốn RAM và chậm.

2.StringBuilder & StringBuffer

Cả hai đều là lớp hỗ trợ thao tác với chuỗi có thể thay đổi (mutable), nhằm giải quyết hạn chế của lớp String (immutable).

Nên dùng khi:

- Bạn cần **nối chuỗi nhiều lần** (trong vòng lặp, xử lý văn bản).
- Muốn tăng hiệu suất và **tiết kiệm bộ nhớ** hơn so với dùng String.

```
StringBuilder sb = new StringBuilder("Hello");
```

Các phương thức thường dùng:

Phương thức	Mô tả	Ví dụ
append(String)	Nối thêm vào cuối chuỗi	sb.append("abc")
insert(int, String)	Chèn chuỗi tại vị trí chỉ định	sb.insert(0, ">>> ")
delete(start, end)	Xóa chuỗi con	sb.delete(2, 4)
replace(start, end, str)	Thay thế chuỗi con	sb.replace(0, 4, "Hi")
reverse()	Đảo ngược chuỗi	sb.reverse()

// Dùng String (chậm)

```
String s = "";  
for (int i = 0; i < 10000; i++) {  
    s = s + i;  
}
```

// Dùng StringBuilder (nhanh)

```
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < 10000; i++) {  
    sb.append(i);  
}
```

3. Khi nào dùng cái nào

- Cần thao tác với chuỗi nhiều lần (nối, xóa, chèn) : **StringBuilder**
- Đang lập trình đa luồng, cần an toàn dữ liệu: **StringBuffer**
- Chỉ làm việc với chuỗi tĩnh, không thay đổi: **String**

#47. Bài Tập Lab 03

Yêu cầu: Viết chương trình Java thực hiện các bước sau:

- Nhập vào tên và điểm của 3 học sinh.
- Lưu tên vào mảng String[], điểm vào mảng double[].
- In ra danh sách học sinh và điểm tương ứng.
- Tính và in ra điểm trung bình của cả lớp.
- In ra học sinh có điểm từ 8 trở lên.

Ví dụ: Output

Nhập tên học sinh 1: Nam

Nhập điểm: 8.5

Nhập tên học sinh 2: An

Nhập điểm: 6

Nhập tên học sinh 3: Huy

Nhập điểm: 9

Danh sách học sinh:

Nam - 8.5

An - 6.0

Huy - 9.0

Điểm trung bình: 7.83

Học sinh có điểm ≥ 8 :

Linh - 8.5

Huy - 9.0

#48. Chữa Bài Tập Lab 03

//todo

@hoidanit

Chapter 6: Lập trình hướng đối tượng cơ bản (OOP)

Tìm hiểu và nắm vững các kiến thức trọng tâm nhất của lập trình hướng đối tượng (Object Oriented Programming)

#49. Tổng quan về chapter 6

//todo

#50. Lập trình hướng đối tượng (Object Oriented Programming - OOP) là gì ?

1. Khái niệm

Lập trình hướng đối tượng (OOP - Object-Oriented Programming) là một phương pháp lập trình dựa trên mô hình các "đối tượng" (object) trong thế giới thực.

Thay vì chỉ tập trung vào các hàm hay quy trình (như lập trình hướng thủ tục), OOP tổ chức chương trình thành các lớp (class) và đối tượng (object) tương tác với nhau.

Tác dụng: OOP giúp viết code rõ ràng, dễ quản lý, dễ mở rộng, và gần gũi với cách con người suy nghĩ trong cuộc sống thực.

Ví dụ: shopee

2. Đặc điểm

- **Đóng gói (Encapsulation)** – gom dữ liệu và các hành vi liên quan vào cùng một đối tượng.
- **Kế thừa (Inheritance)** – cho phép một lớp kế thừa thuộc tính/phương thức từ lớp khác.
- **Đa hình (Polymorphism)** – cùng một phương thức nhưng thực hiện khác nhau tùy đối tượng.
- **Trừu tượng (Abstraction)** – che giấu chi tiết phức tạp, chỉ hiển thị phần quan trọng.

#51. Khái niệm Class và Object

Để mô phỏng "real world", chúng ta cần các "đối tượng" (target)

Đối với 1 sinh vật/đồ vật trong thế giới thực, điều có điều miêu tả bởi 2 thứ:

- Đặc điểm (thuộc tính)
- Hành vi đặc trưng của nó

Ví dụ:

Miêu tả con sư tử: (sinh vật)

- **Đặc điểm** (thuộc tính):

- + Có lông
- + thuộc họ mèo

- **Hành vi:**

- + đi săn mồi

Miêu tả 1 bạn sinh viên: (sinh vật)

- **Đặc điểm:**

- + có mã số sinh viên
- + có địa chỉ sinh sống
- + có tên lớp học

- **Hành vi:**

- + ôn tủ cho qua môn
- + ngủ nướng qua ngày
- + cuối tuần đi chơi

Miêu tả 1 cái máy tính (đồ vật)

- **Đặc điểm:**

- + có dung lượng ổ cứng (SSD/HDD)
- + có dung lượng bộ nhớ (RAM)

- **Hành vi:**

- + có thể on/off/restart

=> Sử dụng "đối tượng" để mô hình hóa.

Tuy nhiên, chúng ta không "miêu chỉ chi tiết", chỉ miêu tả "đối tượng tổng quát" (đây gọi là **tính trừu tượng hóa/abstraction**)

Ví dụ, thay vì miêu tả cụ thể sinh viên A, B, C, D ... chúng ta gọi chung là sinh viên:

Đối tượng tổng quát được gọi là **Class** (Lớp), còn mỗi sinh viên cụ thể gọi là **Object** (đối tượng).

Ví dụ: **class SinhVien** gồm:

- **Thuộc tính** (đặc điểm)

+ Tên

+ mã số sinh viên

- **Hành vi:**

+ ăn/ngủ/nghỉ/ôn thi

=> Để có tính tái sử dụng code cao, **OOP chỉ "định nghĩa" lớp tổng quát.**
còn nếu muốn có chi tiết => chúng ta tự tạo ra

Ví dụ, để tạo ra sinh viên A thì:

+ gán Tên = A

+ gán mã số sinh viên = 1234

Tương tự, tạo ra sinh viên B thì:

+ gán Tên = B

+ gán mã số sinh = 6789

...

Khái niệm:

- **Class** chính là "đối tượng tổng quát" của real-word được mô phỏng (class là bản thiết kế, mô tả khái quát)

- **Object** là các thực thể chi tiết của class đấy (instance)

#52. Bài Tập tạo Class/Object

Khái niệm:

- Class chính là "đối tượng tổng quát" của real-word được mô phỏng
- Object là các thực thể chi tiết của class đấy

Ví dụ về class:

```
public class Student {  
    String name;  
    int age;  
  
    public void learnJava(){  
        System.out.println("Learn Java Core with @hoidanit");  
    }  
}
```

Làm sao để tạo ra các đối tượng Student cụ thể:

```
public static void main(String[] args) {  
    Student a = new Student();  
    a.age = 20;  
    a.name = "Hỏi Dân IT";  
  
    System.out.println("Student a with name = " + a.name + " and age = " + a.age);  
}
```

Ở block code trên thì:

- Student là tên Class

a là Object, một "đối tượng" chi tiết của class Student (instance) /1 bản thể/ 1 bản sao

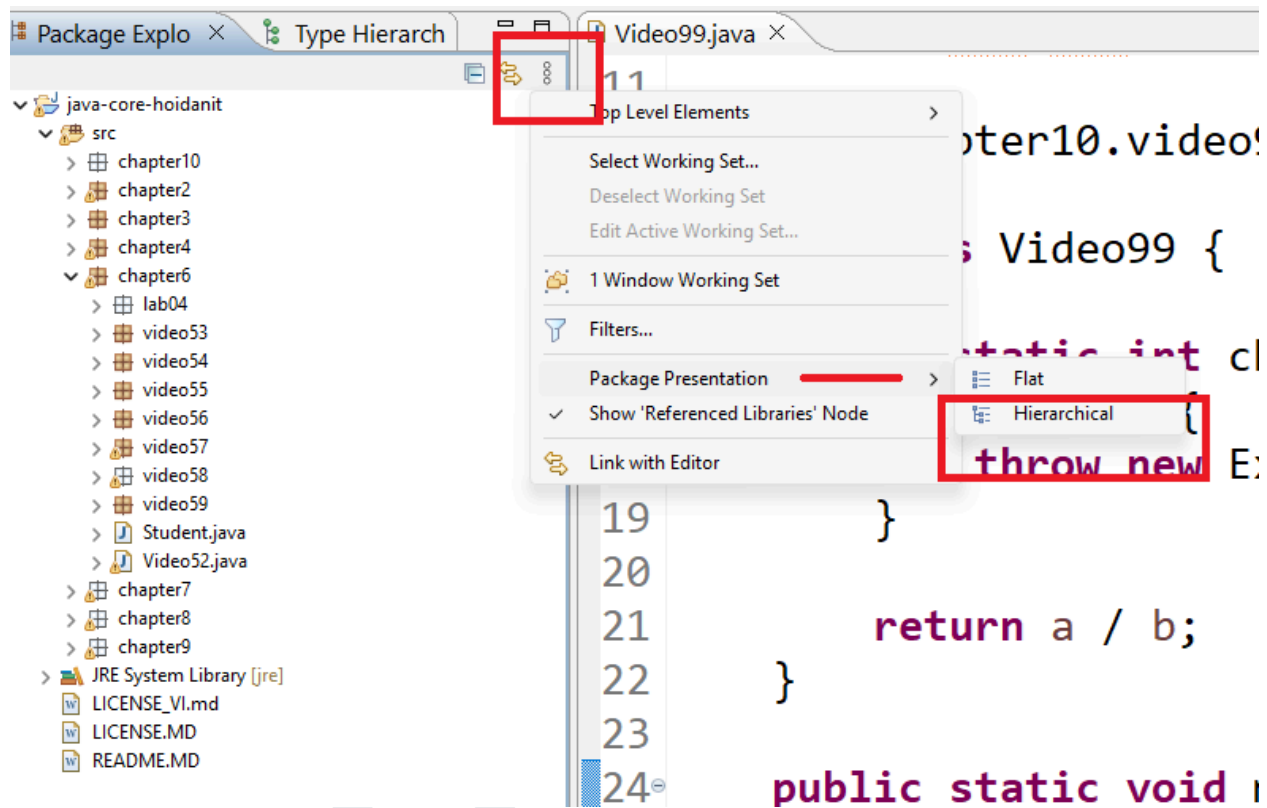
a được tạo ra thông qua keyword "new" => có thể tạo ra bao nhiêu đối tượng tùy thích (từ 1 class ban đầu)

a được gán giá trị thông qua "dot.attribute"

#52.1 Hiện thị Package Collapse (Extra)

Từ video 53, mình sẽ dùng package để code, tương ứng với từng video.

Vì vậy, trong STS, các bạn chọn tính năng hiển thị “Hierarchical” như hình bên dưới nhé



#53. Thuộc tính và Phương thức (Fields & Methods)

//bổ trợ: như thế nào là 1 method (function) với lập trình java

Mục tiêu:

- Hiểu rõ thuộc tính (fields) dùng để lưu trữ dữ liệu cho object
- Biết định nghĩa và sử dụng phương thức (methods) để biểu diễn hành vi
- Biết cách gọi phương thức từ object

1. Thuộc tính (fields)

Định nghĩa: Field là biến khai báo trong class, dùng để lưu thông tin riêng của mỗi object.

Bước 1: khai báo

```
public class Student {  
    String name;  
    int age;  
}
```

Bước 2: gán giá trị

```
Student s = new Student();  
s.name = "hoidanit";  
s.age = 18;
```

Lưu ý: Mỗi object có bộ dữ liệu riêng → không ảnh hưởng nhau

2. Phương thức (Methods) - Hành vi của Object

Khái niệm: Phương thức là hành động mà đối tượng có thể thực hiện.

Nó giống như "hành vi" của đối tượng trong đời thực: học sinh thì có study(), nhân viên thì work(), xe thì run().

Bước 1: khai báo

```
void study() {  
    System.out.println(this.name + " is studying.");  
}
```

Bước 2: gọi phương thức

```
Student s1 = new Student();  
s1.study();
```

Lưu ý:

- Phương thức được gọi bằng cú pháp: `objectName.methodName()`
- Trong method, có thể dùng `this.name` hoặc `name` tùy bối cảnh (chi tiết về `this` học sau)

#54. Bài Tập Fields/Methods

Mục tiêu:

- Áp dụng kiến thức về thuộc tính (fields) và phương thức (methods)
- Hiểu cách gán dữ liệu cho object
- Biết cách in thông tin ra màn hình thông qua method

Yêu cầu: Hãy viết chương trình Java theo yêu cầu sau:

Bước 1:

Tạo class **Employee** với 2 thuộc tính sau:

- String **name**: tên nhân viên
- double **salary**: mức lương

Bước 2:

Thêm 1 phương thức **showInfo()** để in ra thông tin nhân viên theo định dạng:

Name: [Tên nhân viên]

Salary: [Mức lương]

Bước 3:

Trong class Main, hãy:

- **Tạo 2 đối tượng Employee** với dữ liệu khác nhau
- **Gọi phương thức showInfo()** để hiển thị thông tin từng nhân viên

#55. Constructor – Hàm tạo trong Java

Mục tiêu:

- Hiểu constructor là gì và tại sao cần dùng
- Biết cách tạo constructor mặc định và constructor có tham số
- Biết cách khởi tạo object và truyền giá trị ngay lập tức khi tạo

Vấn đề gặp phải: khi tạo object, cần gán giá trị từng dòng riêng lẻ

```
Student s = new Student();  
s.name = "hoidanit";  
s.age = 18;
```

1. Constructor là gì ?

- Constructor là một **hàm đặc biệt được gọi tự động** khi object được tạo.
- Dùng để **khởi tạo giá trị ban đầu** cho object.
- Constructor "phải có tên giống với class", và không thể có "return type"

Tất cả các class đều cần có "hàm tạo". Nếu bạn không tạo => java sẽ tự động tạo cho bạn (không có tham số đầu vào)

Ví dụ:

```
public Student (String name, int age){  
    this.name = name;  
    this.age = age;  
}
```

2. Keyword this

- Dùng để "tham chiếu" tới giá trị của instance/object hiện tại (tương tự con trỏ của C)

//tạo mới object với hàm tạo

#56. Bài Tập Constructor (Hàm tạo)

Viết **class Product** gồm:

- Thuộc tính: name (String), price (double)
- Constructor có 2 tham số: gán giá trị ngay khi tạo object
- Phương thức display() để in thông tin

Lưu ý: 1 class có thể có nhiều hàm tạo với tên giống nhau (khác số lượng tham số). Khi chạy, Java sẽ dựa vào số lượng tham số để lựa chọn constructor phù hợp

Gợi ý:

```
Product p1 = new Product("Laptop", 1500.0);
```

```
Product p2 = new Product("Smartphone", 900.0);
```

Kết quả mong muốn:

Laptop - 1500.0

Smartphone - 900.0

#57. Package và Access Modifier

Mục tiêu:

- Hiểu khái niệm **package** và biết cách tổ chức file Java theo package
- Biết 4 loại **Access Modifier** trong Java: **private, default, protected, public**
- Phân biệt được phạm vi truy cập tương ứng giữa các lớp và gói (package)
- Biết cách viết chương trình có chia package và kiểm soát quyền truy cập phù hợp

1. Package – Cấu trúc tổ chức chương trình

package là cách tổ chức các lớp Java theo nhóm chức năng, giống thư mục.

Mỗi file Java chỉ thuộc 1 package duy nhất

Mục đích:

- Quản lý code gọn gàng hơn
- Hạn chế truy cập từ nơi không liên quan
- Dễ mở rộng hệ thống

//todo: lấy ví dụ (Tên package là tên thư mục chứa file Java)

2. Access Modifier – Phân quyền truy cập

Access Modifier giúp kiểm soát ai có thể truy cập vào:

- Class
- Thuộc tính (fields)
- Phương thức (methods)

4 mức độ truy cập trong Java:

Access Modifier	Cùng class	Cùng package	Package khác (kế thừa)	Package khác
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

private: Chỉ truy cập được bên trong chính class

default : Truy cập trong cùng package (nếu không khai báo gì)

protected: Truy cập được trong cùng package hoặc class con

public: Ai cũng truy cập được

#58. Bài Tập Access Modifier

//todo: giải thích phần default

Yêu cầu:

Tạo 2 package: **model** và **test**

Trong package **model**:

- Tạo class Employee
- Khai báo các thuộc tính sau:
private String name
int age
protected String department
public double salary

Trong package **test**:

Tạo object Employee

In thử từng thuộc tính → xác định thuộc tính nào truy cập được, thuộc tính nào bị lỗi

#59. Encapsulation – Tính đóng gói trong Java

Mục tiêu:

- Hiểu khái niệm Encapsulation (Đóng gói)
- Biết lý do tại sao không nên khai báo biến là public
- Biết cách dùng getter và setter để kiểm soát truy cập đến dữ liệu
- Viết được class với thuộc tính private và method public

1. Encapsulation

Encapsulation là kỹ thuật giấu dữ liệu nội bộ (biến) của class và chỉ cho phép **truy cập thông qua các phương thức công khai** (public methods).

Tác dụng:

- Bảo vệ dữ liệu khỏi bị thay đổi trực tiếp từ bên ngoài
- Giới hạn phạm vi truy cập, ngăn sửa sai lệch
- Cho phép kiểm tra hợp lệ khi gán giá trị

2. Cách sử dụng

- Biến phải là private
- Dùng **getter để xem dữ liệu**
- Dùng **setter để gán dữ liệu** có kiểm soát

Ví dụ:

```
public String getName() {  
    return name;  
}  
  
public void setName(String name) {  
    this.name = name;  
}
```

#60. Bài Tập Lab 04

Tạo package **lab04**

Tạo 2 packages con:

model: chứa class **Employee**

video60: chứa hàm main() để chạy chương trình

Class Employee chứa các thuộc tính sau: (**private**)

- String **name**
- int **age**
- String **department**
- double **salary**
- String **company**

- **Viết hàm tạo Constructor** có 4 tham số: name, age, department, salary
- Viết Getter/Setter
- Viết method:
void **showInfo()**: in thông tin nhân viên

void **raiseSalary**(double amount): tăng lương nếu amount > 0. Tăng 15%

Hàm main:

- Tạo 2 object Employee với thông tin khác nhau (dùng constructor có tham số)
- Gọi showInfo() để in thông tin
- Gọi raiseSalary() để tăng lương
- In lại thông tin sau khi tăng

Ví dụ về output:

== BEFORE RAISE ==

Name: hoidanit

Age: 25

Department: Sales

Salary: 8000.0

Company: ABC Corporation

Name: eric

Age: 30

Department: IT

Salary: 10000.0

Company: ABC Corporation

== AFTER RAISE ==

Name: hoidanit

Age: 25

Department: Sales

Salary: 9200.0

Company: ABC Corporation

Name: eric

Age: 30

Department: IT

Salary: 11500.0

Company: ABC Corporation

#61. Chữa Bài Tập Lab 04

//todo

@hoidanit

Chapter 7: Kế thừa (Inheritance)

Tìm hiểu về tính kế thừa trong Java

#62. Tổng quan về chapter 7

//todo

#63. Kế thừa và từ khóa extends

Mục tiêu:

- Hiểu được **khái niệm kế thừa** trong lập trình hướng đối tượng.
- Biết cú pháp sử dụng từ khóa **extends** trong Java.
- Phân biệt được **quan hệ "is-a"** giữa lớp cha và lớp con.
- Biết cách tạo lớp con kế thừa phương thức từ lớp cha.

1. Kế thừa là gì?

- Là cơ chế cho phép một class mới **tái sử dụng** các thuộc tính và phương thức của class cũ.
- Giúp giảm trùng lặp mã và dễ bảo trì.

"IS-A" (là một loại) là mối quan hệ logic giữa một class con và một class cha.

Nếu bạn có thể nói rằng "class con là một loại class cha", thì đó là mối quan hệ IS-A – bạn nên dùng kế thừa.

2. Cú pháp kế thừa với extends

```
class Cha {  
    // thuộc tính và phương thức  
}
```

```
class Con extends Cha {  
    // có thể thêm hoặc ghi đè phương thức  
}
```

3. Quy tắc kế thừa

- Class con sẽ **kế thừa tất cả** thuộc tính và phương thức của class cha (**trừ private**).
- Java chỉ hỗ trợ **kế thừa đơn** – một class chỉ có thể kế thừa từ một class cha.
- Có thể mở rộng hành vi của lớp cha bằng cách **thêm hoặc ghi đè** phương thức.

#64. Bài Tập Kế Thừa

Yêu cầu:

Tạo một class **Vehicle** với phương thức **start()**, in ra "Vehicle is starting...".

Tạo class **Car** kế thừa từ **Vehicle**, thêm phương thức **honk()** in ra "Car is honking...".

Trong hàm main, tạo đối tượng Car, gọi cả **start()** và **honk()**.

#65. Từ khóa **super** trong constructor và phương thức

Mục tiêu:

- Hiểu **super** dùng để làm gì trong Java.
- Biết cách dùng **super()** để gọi constructor lớp cha.
- Biết cách dùng **super.method()** để gọi lại phương thức lớp cha khi bị ghi đè.

1. **super** là gì?

super đại diện cho **lớp cha** trong Java.

Dùng trong **constructor** hoặc **method** của lớp con để:

- Gọi constructor của lớp cha → **super(...)**
- Gọi phương thức của lớp cha → **super.method(...)**

2. Gọi constructor lớp cha bằng **super()**/ Gọi phương thức lớp cha bằng **super.method()**

Khi nào cần?

- Khi **lớp cha có constructor có tham số**, thì lớp con phải gọi constructor đó bằng **super(...)**.
- **super(...)** luôn nằm dòng đầu tiên trong constructor lớp con.

Ví dụ:

```
class Person {  
    String name;  
    int age;  
}  
class Student extends Person {  
    String school;  
}
```

#66. Bài Tập sử dụng Super

Yêu cầu 1 – Gọi constructor lớp cha bằng super()

- Tạo lớp **Vehicle** có 2 thuộc tính: **brand, year**.
- Tạo constructor có tham số, viết getter/setter

Tạo lớp **Car** kế thừa **Vehicle**, thêm thuộc tính **type**.

Trong constructor Car, dùng super() . viết setter/getter

Tạo mới đối tượng Car, và in ra thông tin:

Vehicle: Toyota, 2022

Type: SUV

Yêu cầu 2 – Gọi phương thức lớp cha bằng super.method()

Trong lớp **Vehicle**, tạo phương thức **printInfo()** in brand + year.

Trong **Car**, ghi đè **printInfo()**, nhưng vẫn gọi lại super.**printInfo()** rồi in thêm type.

Kết quả mong đợi:

Vehicle: Toyota, 2022

Type: SUV

#67. Ghi đè phương thức (Method Overriding)

Mục tiêu:

- Hiểu rõ khái niệm **method overriding** (ghi đè).
- Biết cách sử dụng annotation **@Override** để đảm bảo ghi đè đúng cách.

1. Ghi đè phương thức là gì?

Ghi đè (Overriding) xảy ra khi lớp con định nghĩa lại một phương thức đã tồn tại trong lớp cha:

- Cùng tên
- Cùng kiểu trả về
- Cùng danh sách tham số

Khi đối tượng thuộc lớp con, phương thức của lớp con sẽ được gọi thay vì lớp cha.

//todo: lấy ví dụ

Annotation **@Override** giúp trình biên dịch:

- Báo lỗi nếu bạn ghi đè sai cú pháp (ví dụ: sai tên hàm, sai tham số).
- Gợi nhắc rõ ràng rằng đây là hành vi ghi đè.

#68. Bài Tập Method Overriding

Yêu cầu:

Tạo class **Employee**, gồm:

- String **name**;
- int **id**;
- double **salary**;
- method: double **calculateSalary()**{ } //return salary
- Viết getter/setter

Tạo class **Manager** kế thừa **Employee**:

- Thêm thuộc tính String **department**
- Ghi đè method, đồng thời tính mức lương mới = 110% (salary)

#69. Nạp chồng phương thức (Method Overloading)

Mục tiêu:

- Hiểu rõ khái niệm **method overloading** (nạp chồng phương thức).
- Biết phân biệt **overloading** và **overriding**.
- Áp dụng được overloading để tạo nhiều cách gọi hàm tiện lợi cho người dùng.

1.Method Overloading là gì?

Là việc định nghĩa nhiều phương thức trùng tên trong cùng một lớp, nhưng:

- Khác số lượng tham số, hoặc
- Khác kiểu dữ liệu tham số

Java sẽ quyết định gọi phương thức nào dựa trên kiểu tham số truyền vào.

Ví dụ:

```
int add(int a, int b) {  
    return a + b;  
}
```

```
double add(double a, double b) {  
    return a + b;  
}
```

Lưu ý: Không thể overloading chỉ bằng cách thay đổi kiểu trả về:

```
int add(int a, int b) { ... }
```

```
double add(int a, int b) { ... } // ❌ Lỗi – không phân biệt được
```

2. So sánh Overloading và Overriding

Tiêu chí	Overloading	Overriding
Xảy ra ở đâu?	Cùng một lớp	Giữa lớp cha và lớp con
Tên phương thức	Giống nhau	Giống nhau
Tham số	Khác nhau	Giống nhau
Trả về	Có thể khác nhau	Phải giống hoặc kiểu con
Quyết định gọi lúc	Biên dịch (compile time)	Chạy (runtime)

#70. Bài Tập Method Overloading

Yêu cầu:

Tạo class **CustomerPrinter** có 3 phương thức **print()**:

print(String name)

print(String name, int age)

print(String name, int age, String city)

Tạo đối tượng trong main, gọi lần lượt 3 phương thức để in thông tin.

#71. Bài Tập Lab 05

Yêu cầu:

Tạo class **Person**:

- Thuộc tính: **name, age**
- Constructor có tham số, viết getter/setter
- Phương thức introduce() in ra thông tin cơ bản.

Tạo class **Employee** kế thừa từ Person:

- Thêm thuộc tính: **employeeId, salary**
- Constructor có tham số, gọi super(name, age) để khởi tạo lớp cha.
- Ghi đè introduce() để in thêm employeeId và salary.
- Viết 2 phương thức calculateSalary():
calculateSalary() không tham số → trả về salary
calculateSalary(double bonus) → trả về salary + bonus (nạp chồng)

Tạo class **Manager** kế thừa từ Employee:

- Thêm thuộc tính: **department**
- Ghi đè introduce() để in thêm department.
- Trong introduce(), gọi lại phương thức lớp cha để tái sử dụng (super.introduce()).

Trong **main()**:

Tạo một Manager tên là "hoidanit", tuổi 25, mã nhân viên "M001", lương 2000, thuộc phòng "Sales".

Gọi introduce() và calculateSalary() (cả 2 phiên bản của calculateSalary).

Kết quả mong đợi:

Name: hoidanit

Age: 25

Employee ID: M001

Salary: 2000.0

Department: Sales

Lương cơ bản: 2000.0

Lương sau thưởng: 2500.0

#72. Chữa Bài Tập Lab 05

//todo

Chapter 8: Đa hình (Polymorphism) và Trừu tượng (Abstraction)

Hiểu và áp dụng tính linh hoạt trong thiết kế chương trình hướng đối tượng thông qua đa hình và trừu tượng hóa

#73. Tổng quan về chapter 8

//todo

#74. Phân biệt Compile-time vs Runtime (Extra)

1.Compile-time (thời điểm biên dịch)

Khái niệm: Là giai đoạn khi mã nguồn được dịch (compile) thành bytecode bởi trình biên dịch Java (javac).

Mọi kiểm tra cú pháp, kiểu dữ liệu, chữ viết sai, method overloading... đều xảy ra lúc này.

Đặc trưng:

- Xác định được kiểu biến, tên hàm, tham số có hợp lệ không.
- Nếu sai → lỗi biên dịch (compile-time error) xảy ra.

Ví dụ:

int age = "twenty"; // ❌ lỗi: gán chuỗi cho biến int

System.out.printl("Hello"); // ❌ lỗi: viết sai `println`

System.out.println(score); // ❌ lỗi: biến score chưa được khai báo

Compile-time errors là lỗi mà Java phát hiện trước khi chạy, do cú pháp sai, kiểu sai, tên sai, hoặc sử dụng sai quy tắc của ngôn ngữ.

2.Runtime (thời điểm chạy)

Khái niệm: Là giai đoạn chương trình đã biên dịch được chạy trên máy ảo Java (JVM).

Các hành vi như method overriding, ép kiểu, instanceof, lỗi logic, lỗi null xảy ra ở thời điểm này.

Đặc trưng:

- Chỉ tại lúc chạy chương trình, JVM mới xác định được object thật sự là của class nào.
- Một số lỗi chỉ xuất hiện khi chạy.

Ví dụ:

```
int a = 10;  
int b = 0;  
int result = a / b; // ❌ lỗi chia cho 0
```

```
Int[] numbers = {1, 2, 3};  
System.out.println(numbers[3]); // ❌ lỗi: chỉ có chỉ số 0-2
```

Bonus: Quá trình bạn thực hiện khi code trong IDE là gì?

Bạn đang làm gì	Giai đoạn trong IDE	Thuộc compile-time hay runtime?
Gõ code và thấy gạch đỏ	IDE kiểm tra cú pháp, kiểu	✅ Compile-time (hỗ trợ sớm)
Nhấn Build/Run	Biên dịch mã với javac	✅ Compile-time
Mở console và thấy kết quả	Chương trình chạy bằng JVM	✅ Runtime
Gặp lỗi `NullPointerException`, ép kiểu sai	Chỉ xuất hiện khi chạy	✅ Runtime error

#75. Tính Đa Hình (Polymorphism)

Mục tiêu:

- Hiểu khái niệm **đa hình (polymorphism)** là gì trong OOP.
- Hiểu và thực hành với **method overloading** và **method overriding**.
- Biết đa hình giúp viết mã dễ mở rộng và tái sử dụng như thế nào.

1. Khái niệm đa hình (Polymorphism)

"Đa hình" = một hành vi có thể có nhiều cách thực hiện khác nhau.

Polymorphism = many forms

Trong Java, có hai loại đa hình chính:

Loại đa hình	Thể hiện qua	Thời điểm quyết định
Đa hình tĩnh	Method Overloading	Compile time
Đa hình động	Method Overriding	Runtime

2. Đa hình tĩnh (Static Polymorphism) – Method Overloading

Định nghĩa: Trong cùng một class, nhiều phương thức có cùng tên nhưng khác tham số (kiểu, số lượng...).

//todo

3. Đa hình động (Dynamic Polymorphism) – Method Overriding

Định nghĩa: Khi lớp con ghi đè phương thức của lớp cha.

Lúc chạy, Java gọi phương thức từ object thực tế, không dựa vào kiểu biến tham chiếu.

//todo

#76. Ứng dụng đa hình thực tế – Upcasting, Downcasting, instanceof

Mục tiêu:

- Hiểu rõ khái niệm **upcasting** và **downcasting**.
- Biết cách dùng **instanceof** để kiểm tra kiểu thực tế của object.

1. Upcasting (Ép kiểu lên)

Một biến kiểu cha (Animal) có thể **tham chiếu** đến object lớp con (Dog)

→ gọi là upcasting (chuyển một object của lớp con thành kiểu của lớp cha)

Ví dụ:

```
Animal a = new Dog( ); // upcasting
```

```
a.makeSound( ); // Gọi phương thức bị Dog hay là Animal ? a có thể gọi wagTail ?
```

```
System.out.println(a.getClass());
```

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
  
    void wagTail() {  
        System.out.println("Wagging tail...");  
    }  
}
```

2. Downcasting – Ép kiểu cha về con

Chỉ dùng khi chắc chắn object thực là lớp con

Ví dụ:

```
Animal a = new Dog(); // upcasting
Dog d = (Dog) a;      // ✅ downcasting hợp lệ
d.bark();              // OK vì a thật sự là Dog
```

```
Animal a = new Animal();
Dog d = (Dog) a;      // ❌ Lỗi tại runtime
d.bark();
```

3. instanceof – Kiểm tra kiểu thật sự

- Tránh lỗi khi downcast sai
- Giúp xử lý chính xác theo loại object thực tế

Ví dụ:

```
if (a instanceof Dog) {
    Dog d = (Dog) a;
    d.bark();
}
```

#77. Method Resolution của Java (Extra)

Java gọi phương thức dựa trên object thực tế, không dựa vào kiểu khai báo của biến.

```
Animal a1 = new Animal();
```

```
Animal a2 = new Dog();
```

```
a1.makeSound(); // Animal makes a sound
```

```
a2.makeSound(); // Dog barks
```

Biến a2 là kiểu Animal, nhưng object thực tế là Dog, nên gọi Dog.makeSound().

#78. Bài Tập về tính Đa Hình

Yêu cầu

Tạo class **Vehicle** có:

- Thuộc tính: String brand
- Phương thức: void **start()** in ra "Vehicle is starting..."

Tạo các class kế thừa:

- **Car** extends Vehicle – override start(): "Car [brand] is starting with key..."
- **Motorbike** extends Vehicle – override start(): "Motorbike [brand] is kick-starting..."
- **Bus** extends Vehicle – override start(): "Bus [brand] is starting with air brakes..."

Trong main():

- Tạo mảng **Vehicle[]** chứa các object **Car, Motorbike, Bus** (dùng upcasting).
- Duyệt qua **mảng** và gọi **start()** cho từng phần tử.

Output:

Car toyota is starting with key...

Bus vin is starting with air brakes...

Motorbike honda is kick-starting...

#79. Abstract Class

1. Khái niệm Abstract Class là gì?

- **Abstract class** (lớp trừu tượng) là lớp **không thể tạo đối tượng trực tiếp**.
- Nó có thể chứa **phương thức trừu tượng** (không có thân hàm) và **phương thức thường** (có thân hàm).
- Được dùng làm **lớp cha**, để định nghĩa những hành vi chung mà **các lớp con phải triển khai**.

Tại sao dùng Abstract Class?

- Để **định nghĩa cấu trúc chung** cho nhiều lớp liên quan.
- Giúp **ép buộc** các lớp con phải cài đặt một số hành vi nhất định.
- Cho phép **tái sử dụng** mã nguồn dùng chung trong các lớp con.

Ví dụ: Một lớp Animal có thể định nghĩa makeSound() là trừu tượng, nhưng sleep() là hành vi có sẵn cho tất cả loài vật.

2. Cú pháp

- Dùng từ khóa **abstract** trước lớp hoặc phương thức.
- Nếu lớp có ít nhất một phương thức trừu tượng → bắt buộc là abstract class.

```
abstract class Animal {  
    abstract void makeSound ( ); // phương thức trừu tượng  
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

3. So sánh với class thông thường

Tiêu chí	Abstract class	Class thường
Khả năng khởi tạo từ khóa new	✗ Không được	✓ Được
Chứa phương thức abstract	✓ Có thể	✗ Không được
Chứa phương thức bình thường	✓ Có thể	✓ Có thể
Mục đích sử dụng	Định nghĩa khung sườn chung cho các lớp con	Chứa logic cụ thể, triển khai đầy đủ
Có thể chứa thuộc tính	✓ Có thể	✓ Có thể
Ví dụ sử dụng	Làm lớp cha cho các loại động vật, phương tiện, nhân viên...	Lớp cụ thể như Dog, Car, Manager, Product...

#80. Bài Tập Abstract Class

Yêu cầu:

Tạo **abstract class Employee** trong package, gồm:

Thuộc tính:

- String **name**
- String **department**
- Constructor khởi tạo các thuộc tính trên
- Phương thức getter/setter cho name và department
- Phương thức **abstract double calculateSalary()**
- Phương thức void **printInfo()** in thông tin chung của nhân viên (name, department)

Tạo các lớp con kế thừa:

Class **FullTimeEmployee** gồm:

Thuộc tính:

- double **monthlySalary**
- Ghi đè **calculateSalary()** trả về monthlySalary
- In thêm "Full-time" trong **printInfo()**

Class **PartTimeEmployee** gồm:

Thuộc tính:

- int **hoursWorked**
- double **hourlyRate**
- Ghi đè **calculateSalary()** trả về hoursWorked * hourlyRate
- In thêm "Part-time" trong printInfo()

Trong lớp main:

- Tạo danh sách **Employee** bao gồm cả **FullTimeEmployee** và **PartTimeEmployee**
- Duyệt danh sách, in thông tin và lương từng nhân viên

Ví dụ output:

Name: Hoidanit | Department: IT | Type: Full-time

Salary: 3000.0

Name: Eric | Department: HR | Type: Part-time

Salary: 1200.0

#81. Interface

1. Khái niệm Interface là gì?

- **Interface là một khuôn mẫu** (hợp đồng) định nghĩa các **hành vi** mà một lớp phải thực hiện.
- Nó **chỉ khai báo phương thức**, không chứa cài đặt cụ thể (trừ khi dùng default hoặc static method).
- Interface giúp **tách phần định nghĩa hành vi** ra khỏi phần triển khai cụ thể.

Interface = “Tôi đảm bảo là tôi sẽ làm những việc sau, nhưng cách làm thì sẽ do từng lớp tự xử lý.”

2. Cú pháp khai báo Interface

- Tất cả các phương thức trong interface mặc định là **public abstract** (dù bạn không cần ghi rõ).
- Các biến trong interface mặc định là **public static final** (tức là hằng số).

```
public interface Vehicle {  
    void start();    // khai báo hành vi  
    void stop();  
}
```

#82. Bài Tập Interface

Yêu cầu:

Tạo một **interface** tên là **Playable** có:

- void **play**();
- void **pause**();

Tạo các lớp sau, thực hiện interface Playable:

- **Class Song**: đại diện cho một bài hát
Thuộc tính: String **title**, String **artist**
Triển khai **play**() → In: "Playing song: [title] by [artist]"
Triển khai **pause**() → In: "Pausing song: [title]"
- **Class Video**: đại diện cho một video
Thuộc tính: String **title**, int **duration**
Triển khai **play**() → In: "Playing video: [title] ([duration] mins)"
Triển khai **pause**() → In: "Pausing video: [title]"

Tạo lớp Main:

- Tạo **mảng** hoặc danh sách **Playable** gồm nhiều Song và Video
Duyệt qua danh sách, gọi **play**() và **pause**() với từng phần tử

Ví dụ về output:

Playing song: Shape of You by Ed Sheeran

Pausing song: Shape of You

Playing video: Inception (148 mins)

Pausing video: Inception

#83. Abstract Class vs Interface (Extra)

1. Abstract class là gì?

Là **một bản thiết kế chưa hoàn chỉnh**. Nó giống như một **lớp cha** có thể chứa sẵn code, nhưng không được tạo đối tượng trực tiếp.

Dùng để:

- Định nghĩa **phần chung** cho các lớp con (có thể có sẵn code).
- Ép buộc các lớp con phải viết thêm phần riêng (qua abstract method).

Ví dụ:

```
abstract class Animal {  
    String name;  
  
    void sleep() {  
        System.out.println(name + " is sleeping");  
    }  
  
    abstract void makeSound(); // bắt buộc lớp con phải viết  
}
```

2. Interface là gì?

Là **một hợp đồng**. Nó chỉ định rằng: "Lớp nào ký hợp đồng này thì phải làm đúng như tôi khai báo."

Dùng để:

- **Mô tả hành vi** (ví dụ: có thể bay, có thể chạy).
- Cho phép **nhiều lớp** cùng chia sẻ một khả năng mà **không cần quan hệ cha-con**.

Từ Java 8, **interface** có thể chứa **default** method (có sẵn code), nên càng linh hoạt hơn.

Ví dụ:

```
interface Flyable {  
    void fly();  
  
    default void land() {  
        System.out.println("Landing safely...");  
    }  
}
```

2. Phân biệt Abstract class vs Interface

So sánh	abstract class	interface (Java 8 trở lên)
Có tạo object được không?	✗ Không	✗ Không
Có biến thường không?	✓ Có (biến instance như name, age,...)	✗ Không (chỉ có public static final – hằng số)
Có phương thức có code không?	✓ Có thể có	✓ Có (qua default, static) từ Java 8
Có phương thức abstract không?	✓ Có	✓ Có
Có constructor không?	✓ Có (dùng để gán giá trị thuộc tính)	✗ Không
Một class kế thừa được mấy cái?	✗ Chỉ 1 abstract class duy nhất	✓ Nhiều interface cùng lúc
Dùng khi nào?	Khi các lớp có quan hệ “cha-con”, chia sẻ code	Khi các lớp cùng có khả năng, không cần quan hệ

#84. Bài Tập Lab 06

Yêu cầu:

Tạo **abstract** class **Person**:

- Thuộc tính: String **name**, int **age**
- Constructor khởi tạo, getter/setter

Phương thức:

- void **introduce**() → in ra "My name is [name], I am [age] years old."
- abstract void **work**()

Tạo **interface** **Trainable**:

- Phương thức:
void **attendTraining**(String topic)
default void feedback() → in "Thanks for the training session."

Tạo 2 lớp con kế thừa **Person**:

a. **Engineer** implements **Trainable**

- Thuộc tính riêng: String **specialty**
- Ghi đè **work**() → in: "Engineer [name] is working on [specialty] project."
- Ghi đè **attendTraining**(String topic) → in: "Engineer [name] is attending training on [topic]."

b. **Manager** implements **Trainable**

- Thuộc tính riêng: int **teamSize**
- Ghi đè **work**() → in: "Manager [name] is managing a team of [teamSize] people."
- Ghi đè **attendTraining**(String topic) → in: "Manager [name] is attending management training on [topic]."

Trong lớp Main:

Tạo mảng Person[] chứa cả Engineer và Manager (dùng upcasting)

Viết vòng lặp duyệt mảng:

- Gọi **introduce()** và **work()** (thể hiện đa hình động)
- Kiểm tra nếu phần tử là Trainable → ép kiểu (downcasting) và gọi **attendTraining() + feedback()**

Ví dụ về output:

My name is Alice, I am 25 years old.

Engineer Alice is working on AI project.

Engineer Alice is attending training on Cloud Computing.

Thanks for the training session.

My name is Bob, I am 40 years old.

Manager Bob is managing a team of 10 people.

Manager Bob is attending management training on Leadership.

Thanks for the training session.

#85. Chữa Bài Tập Lab 06

//todo

Chapter 9: Enum và Nested Class

Hiểu và áp dụng enum để biểu diễn các hằng số có cấu trúc, đồng thời nắm vững các loại nested class (inner, static, local, anonymous) để tổ chức mã nguồn gọn gàng và linh hoạt

#86. Tổng quan về chapter 9

//todo

#87. Từ khóa final và static (Extra)

Mục tiêu:

- Hiểu và phân biệt được **final** và **static**.
- Biết cách sử dụng **final** để khai báo hằng số, phương thức không được ghi đè, và lớp không được kế thừa.
- Biết cách sử dụng **static** để khai báo thành phần thuộc về lớp thay vì đối tượng.

1. final là gì?

final (cuối cùng), là cách khai báo hằng số trong java, dùng để ngăn chặn việc thay đổi.

Ứng dụng	Ý nghĩa
Biến (final variable)	Không thể gán lại giá trị sau khi đã gán
Phương thức (final method)	Không thể override trong lớp con
Lớp (final class)	Không thể bị kế thừa

Ví dụ:

```
final int MAX_USERS = 100;  
// MAX_USERS = 200; // Lỗi biên dịch
```

```
final class A { }  
// class B extends A { } // Lỗi vì A là final
```

2. Từ khóa static

static (tĩnh) dùng để đánh dấu thành phần thuộc về lớp thay vì đối tượng cụ thể. Có nghĩa là, bạn thao tác trực tiếp với class, không cần phải thông qua đối tượng

static được sinh ra để biểu diễn những thứ thuộc về "**lớp**", không thuộc về "**object**". Nó giúp tiết kiệm bộ nhớ, giảm lặp lại, và làm rõ ý định chia sẻ dữ liệu.

Ví dụ:

```
class Counter {  
    static int count = 0; // dùng chung cho mọi object  
    int id;  
  
    Counter() {  
        count++;  
        id = count;  
    }  
}
```

3. So sánh final và static

Đặc điểm	final	static
Ngăn thay đổi	✓	✗
Dùng chung giữa các object	✗	✓
Có thể gán lại	✗	✓
Gọi được từ tên lớp	✗	✓

#88. Sử dụng enum thay thế static final

1. Nhược điểm của final static

Ví dụ:

```
public class Role {  
    public static final String ADMIN = "ADMIN";  
    public static final String GUEST = "GUEST";  
    public static final String USER = "USER";  
}
```

String role = Role.ADMIN; //  Dùng đúng qua class

String role = "SUPERADMIN" //  Bạn không thể đảm bảo mọi người trong dự án đều tuân thủ như vậy. (vì bản chất, role là String, gõ code không báo lỗi)

Dùng **static final** là đủ nếu bạn kiểm soát được toàn bộ code. Tuy nhiên, để an toàn tuyệt đối trong team lớn, nên dùng **enum**.

2. enum là gì?

enum (viết tắt của enumeration) là kiểu dữ liệu đặc biệt đại diện cho tập hợp các giá trị cố định đã biết trước.

Ví dụ:

```
enum Role {  
    ADMIN, USER, GUEST  
}  
  
public static void main(String[] args) {  
    Role myRole = Role.ADMIN;  
    if (myRole == Role.ADMIN) {  
        System.out.println("Bạn có toàn quyền truy cập.");  
    }  
}
```

#89. Thêm thuộc tính và phương thức vào enum

Enum là một loại class đặc biệt của Java, vì vậy, bạn có thể:

- Thêm thuộc tính (field)
- Viết constructor (luôn là private)
- Viết phương thức (method), cả getter, override, v.v.

Vi dụ:

```
enum Level {  
    LOW("Thấp"), MEDIUM("Vừa"), HIGH("Cao");  
  
    private final String description;  
  
    Level(String description) {  
        this.description = description;  
    }  
  
    public String getDescription() {  
        return description;  
    }  
}
```

#90. Bài Tập về enum

Yêu cầu:

Tạo enum tên **OrderStatus** gồm:

PENDING("Chờ xác nhận")

SHIPPED("Đang giao hàng")

DELIVERED("Đã giao")

CANCELLED("Đã hủy")

Viết thêm hàm `hasFinal()`:

Trả về true nếu là **DELIVERED** hoặc **CANCELLED**

Viết chương trình cho phép người dùng nhập trạng thái đơn hàng (theo enum), và in mô tả tiếng Việt.

Output mong đợi:

Nhập trạng thái đơn hàng: DELIVERED

Trạng thái: Đã giao

Trạng thái cuối cùng: true

//nhập sai:

Nhập trạng thái đơn hàng: SHIPPING

Trạng thái không hợp lệ!

Gợi ý:

Chuyển chuỗi nhập vào thành enum bằng **OrderStatus.valueOf(input)**

#91. Inner Class và Static Nested Class

1.Inner Class (Lớp lồng nhau không static)

Là lớp được khai báo bên trong một lớp khác, không có từ khóa static.

```
class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
    class IDCard {  
        void show() {  
            System.out.println("Thẻ tên của: " + name);  
        }  
    }  
}
```

Đặc điểm:

Gắn liền với một object của outer class:

// Person.IDCard card = new Person.IDCard(); // ❌ Không hợp lệ

Person p = new Person("hoidanit");

Person.IDCard card = p.new IDCard(); // ✅ Phải gắn với p

Có thể truy cập biến và phương thức non-static của outer class (kể cả private)

//todo

2. Static Nested Class (Lớp lồng nhau có static)

Là lớp được khai báo bên trong một lớp khác, với từ khóa static.

Ví dụ:

```
class Outer {  
    static class StaticNested {  
        void sayHi() {  
            System.out.println("Chào từ static nested class!");  
        }  
    }  
}
```

So sánh: Inner Class vs Static Nested Class

Tiêu chí	Inner Class	Static Nested Class
Có từ khóa static	✗ Không	✓ Có
Gắn với object outer	✓ Có	✗ Không
Truy cập non-static outer	✓ Có thể	✗ Không thể
Cách gọi	outer.new Inner()	Outer.StaticNested
Dùng khi	Cần truy cập dữ liệu của outer object	Class chỉ là tiện ích phụ, không cần outer object

#92. Local Class và Anonymous Class

1. Local Class – Lớp cục bộ trong phương thức

Local class là một class có tên, được định nghĩa bên trong một phương thức, khối lệnh, hoặc constructor.

Chỉ có thể sử dụng bên trong nơi nó được khai báo.

Ví dụ:

```
public class Greeting {  
    public void sayHello(String name) {  
        class Formatter {  
            String format() {  
                return "Hello, " + name + "!";  
            }  
        }  
  
        Formatter f = new Formatter();  
        System.out.println(f.format());  
    }  
}
```

2. Anonymous Class – Lớp không tên (vô danh)

Anonymous class là class không có tên, được khai báo và khởi tạo ngay tại chỗ, thường để:

- Override một class
- Triển khai một interface
- Dùng khi bạn chỉ cần class đó một lần duy nhất.

Ví dụ:

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Đang chạy...");  
    }  
};  
task.run();
```

#93. Bài Tập Lab 07

Yêu cầu:

Tạo **enum OrderStatus** gồm các trạng thái:

- PENDING("Chờ xác nhận")
- SHIPPED("Đang giao hàng")
- DELIVERED("Đã giao")
- CANCELLED("Đã hủy")
- Mỗi trạng thái có:

Mô tả tiếng Việt (**description**)

Phương thức `hasFinal()` trả về `true` nếu là `DELIVERED` hoặc `CANCELLED`

Tạo class **Order**, có:

- String **id** (mã đơn hàng)
- OrderStatus **status**

Trong phương thức **processOrder()** của **Order**, hãy:

- Khai báo local **class Logger** để log thông tin đơn hàng (mã đơn + trạng thái).
- Gọi **Logger** để in log khi đơn bắt đầu xử lý.

Inner class CustomerInfo chứa **tên** và **số điện thoại** người đặt hàng.

- Có phương thức **printInfo ()** in thông tin khách hàng kèm trạng thái đơn.

Tạo class **DiscountHelper** là **static nested class** bên trong **Order**.

Có phương thức **calculateDiscount(OrderStatus status)**

Trả về 10% nếu `SHIPPED`, 0% nếu `DELIVERED` hoặc `CANCELLED`, và 5% cho trạng thái khác.

Tạo một Runnable bằng **anonymous class** trong main() để mô phỏng xử lý đơn hàng:
In "Đơn hàng <id> đang được xử lý ở trạng thái: <description>"

Ví dụ:

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Đang chạy...");  
    }  
};  
task.run();
```

Output mong muốn:

[LOG] Xử lý đơn hàng ID: A001 - Trạng thái: Chờ xác nhận

Khách hàng: hoidanit - 0901234567

Trạng thái đơn hàng: Chờ xác nhận

Chiết khấu áp dụng: 5.0%

Đơn hàng A001 đang được xử lý ở trạng thái: Chờ xác nhận

#94. Chữa Bài Tập Lab 07

//todo

#95. Nhìn lại các kiến thức đã học

Phần 1: Lập trình Java Cơ Bản (Java Fundamentals)

Bao gồm:

- **Chapter 2:** Giới thiệu về Java
- **Chapter 3:** Cú pháp cơ bản và biến
- **Chapter 4:** Cấu trúc điều kiện
- **Chapter 5:** Mảng và chuỗi

Phần 2: Lập trình hướng đối tượng (Object Oriented Programming)

Bao gồm:

- **Chapter 6:** Lập trình hướng đối tượng cơ bản
- **Chapter 7:** Tính đóng gói và kế thừa
- **Chapter 8:** Đa hình và trừu tượng
- **Chapter 9:** Enum và Nested Class

Chapter 10: Xử lý ngoại lệ (Exception)

Giới thiệu cách xử lý lỗi trong Java bằng try-catch-finally, throw, throws, và cách tạo ngoại lệ riêng bằng cách kế thừa Exception hoặc RuntimeException, từ đây viết code an toàn hơn

#96. Tổng quan về chapter 10

//todo

#97. Exception và Try/Catch/Finally

Mục tiêu:

- Hiểu rõ **ngoại lệ là gì**, tại sao cần xử lý ngoại lệ
- Biết **khi nào xảy ra ngoại lệ** trong chương trình Java
- Phân biệt cách dùng **try/catch** và **try/catch/finally**

1. Ngoại lệ là gì?

Ngoại lệ (Exception) là các lỗi xảy ra khi chương trình **đang chạy** (runtime).

Khác với lỗi biên dịch (compile-time), ngoại lệ xảy ra khi chương trình đã biên dịch thành công nhưng gặp lỗi trong quá trình thực thi.

Ví dụ:

```
int a = 10 / 0; // Gây ra ArithmeticException: chia cho 0
```

2. Tại sao cần xử lý ngoại lệ?

- Nếu không xử lý, chương trình sẽ bị dừng ngay lập tức khi có lỗi xảy ra.
- Xử lý ngoại lệ giúp:

Giữ cho chương trình tiếp tục chạy ổn định.

Thông báo lỗi rõ ràng cho người dùng hoặc ghi log cho lập trình viên.

Tránh mất dữ liệu hoặc trạng thái không mong muốn.

3.Cấu trúc try/catch và try/catch/finally

Cách 1: try/catch

Dùng khi cần xử lý lỗi đơn giản và tiếp tục chương trình.

Ví dụ:

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Lỗi chia cho 0!");  
}
```

Cách 2: try/catch/finally

Dùng khi cần xử lý lỗi và luôn thực hiện một hành động sau đó, dù có lỗi hay không (ví dụ: đóng file, kết nối, giải phóng tài nguyên).

Ví dụ:

```
try {  
    int result = 10 / 2;  
} catch (ArithmeticException e) {  
    System.out.println("Lỗi chia cho 0!");  
} finally {  
    System.out.println("Khối finally luôn được thực thi.");  
}
```

Đặc điểm	try/catch	try/catch/finally
Có xử lý lỗi không?	✓	✓
Có khối luôn chạy?	✗	✓ (kể cả khi có lỗi hay không)
Dùng khi nào?	Khi chỉ cần xử lý lỗi	Khi cần dọn dẹp tài nguyên, log,...

#98. Java - Try with Resources (Extra)

Tài liệu: <https://www.baeldung.com/java-try-with-resources>

```
//todo
```

#99. Sử dụng throw và throws trong xử lý lỗi

1.throw – Phát sinh ngoại lệ thủ công

Dùng để tạo và ném ra một đối tượng ngoại lệ cụ thể khi phát hiện điều kiện không hợp lệ trong chương trình.

Cú pháp: `throw new ExceptionType("Thông báo lỗi");`

Sau throw phải là một đối tượng thuộc lớp Throwable hoặc các lớp con (Exception, RuntimeException,...)

Ví dụ:

```
public void checkAge(int age) {  
    if (age < 18) {  
        throw new IllegalArgumentException("Chưa đủ 18 tuổi để đăng ký.");  
    }  
}
```

2.throws – Khai báo ngoại lệ của phương thức

Dùng để khai báo rằng một phương thức có thể ném ra ngoại lệ, yêu cầu người gọi phương thức đó phải xử lý hoặc tiếp tục khai báo lại.

Cú pháp thường đi cùng với các ngoại lệ liên quan đến file, IO, hoặc xử lý tài nguyên.

Cú pháp:

```
public void doSomething( ) throws ExceptionType {  
    // code có thể gây ra ExceptionType  
}
```

3. So sánh nhanh throw vs throws

Đặc điểm	throw	throws
Vị trí sử dụng	Trong thân phương thức	Trong phần khai báo đầu của phương thức
Chức năng	Phát sinh một ngoại lệ	Khai báo phương thức có thể phát sinh lỗi
Theo sau là gì	Một đối tượng ngoại lệ	Một hoặc nhiều loại ngoại lệ
Sử dụng cùng lúc được?	Có thể dùng throws để khai báo, và dùng `throw` để phát sinh thực tế	✓

#100. Bài Tập Try/Catch/Throw

Yêu cầu:

Viết một chương trình Java gồm phương thức **checkScore**(int score):

Phương thức này sẽ:

- Nếu score < 0 hoặc score > 10, ném ra (throw) một **IllegalArgumentException** với thông báo "Điểm không hợp lệ: phải từ 0 đến 10".
- Nếu điểm hợp lệ, in ra "Điểm của bạn là: x".

Trong **main**:

- Nhập vào một số nguyên từ bàn phím đại diện cho điểm.
- Gọi phương thức **checkScore(score)** bên trong **try/catch**.
- Nếu có ngoại lệ, bắt (catch) và in ra thông báo lỗi.
- Cuối cùng, trong **finally**, in ra "Kết thúc chương trình."

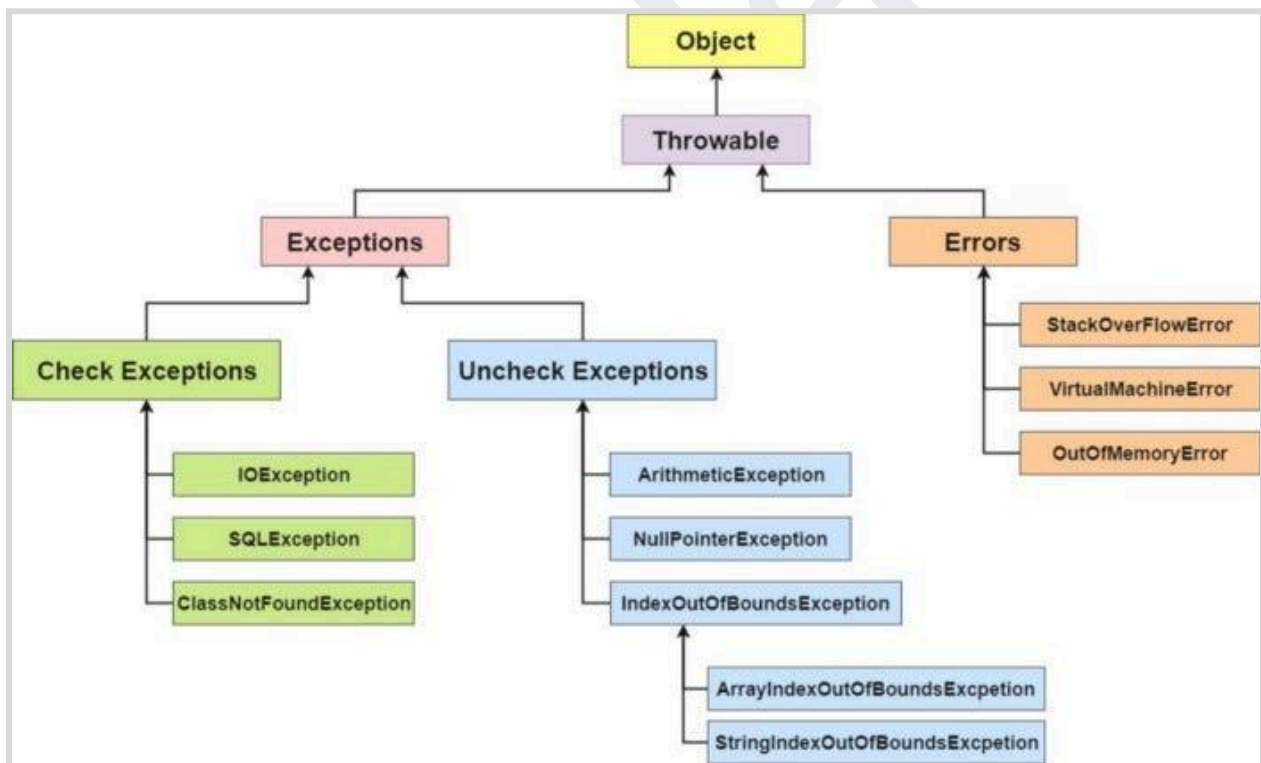
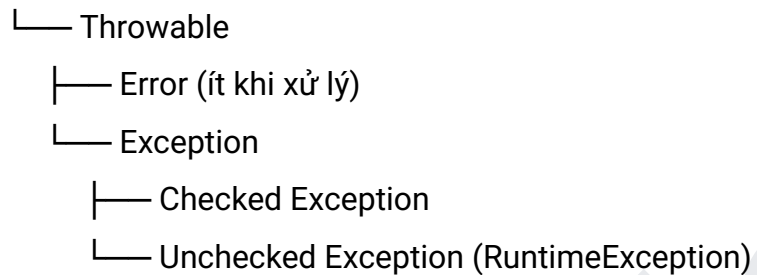
#101. Phân Loại Exception (Extra)

1.Exception là gì? (ôn tập)

- **Exception** là lỗi xảy ra khi chương trình đang chạy (**runtime**).
- Exception là lớp con của **Throwable**.

Cấu trúc kế thừa:

Object



2. So sánh Checked Exception vs Unchecked Exception

Checked Exception:

- Là ngoại lệ **bắt buộc phải xử lý** (bằng try-catch hoặc throws).
- Nếu không xử lý, **chương trình không biên dịch được. (xảy ra tại compile time)**

Ví dụ:

- IOException
- SQLException
- FileNotFoundException

Unchecked Exception:

- Là ngoại lệ **không bắt buộc xử lý**, nhưng nếu không xử lý có thể làm chương trình bị dừng.
- Thường do **lỗi logic của lập trình viên. (xảy ra tại runtime)**

Ví dụ:

- ArithmeticException (chia cho 0)
- NullPointerException (truy cập biến null)
- ArrayIndexOutOfBoundsException (truy cập mảng sai chỉ số)
- NumberFormatException (parse sai kiểu dữ liệu)

#102. Tạo Custom Exception

1. Custom Exception là gì?

- Custom Exception (ngoại lệ tùy chỉnh) là **lớp ngoại lệ do lập trình viên tự định nghĩa**, kế thừa từ **Exception** hoặc **RuntimeException**.
- Dùng khi chương trình có **quy tắc nghiệp vụ riêng** cần thông báo lỗi rõ ràng, thay vì dùng ngoại lệ có sẵn.

2. Cách tạo Custom Exception

Bước 1: Kế thừa lớp Exception hoặc RuntimeException

Kế thừa **Exception** (Checked Exception)

- Ngoại lệ bắt buộc phải xử lý hoặc khai báo throws.

Kế thừa **RuntimeException** (Unchecked Exception)

- Không bắt buộc xử lý, nhưng nếu không bắt sẽ crash runtime.

Bước 2: Tạo constructor

Tạo constructor nhận message, gọi super(message) để truyền thông báo lên lớp cha.

```
//todo
```

#103. Về Xử Lý Exception

Một vài nguyên tắc khuyến khích khi code với Java:

1. Try-with-resources để đóng tài nguyên

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {  
    // đọc file  
}
```

2. Tạo Custom Exception có ý nghĩa nghiệp vụ

Tạo ngoại lệ tùy chỉnh với thông báo rõ ràng giúp:

- Dễ debug.
- Phân biệt được lỗi nghiệp vụ (Business Exception) và lỗi hệ thống (System Exception).

3. Đừng throw Exception chung chung

Tránh viết:

```
throw new Exception("Error");
```

Hãy throw loại ngoại lệ cụ thể hoặc Custom Exception để người gọi biết cách xử lý.

#104. Bài Tập Lab 08

Yêu cầu: Viết chương trình Quản lý đăng ký tài khoản ngân hàng, gồm các tính năng và xử lý lỗi sau:

1. Tạo Custom Exception

- **NegativeBalanceException** (Checked Exception)
Kế thừa Exception.
Dùng khi rút tiền vượt quá số dư.
- **WeakPasswordException** (Unchecked Exception)
Kế thừa RuntimeException.
Dùng khi mật khẩu có độ dài < 6 ký tự.

2. Tạo class BankAccount có:

- **Thuộc tính:**
String username
String password
double balance
- **Constructor:** nhận username, password, balance.
Nếu password < 6 ký tự, ném **WeakPasswordException**.
- Phương thức **withdraw**(double amount)
Nếu amount > balance, ném **NegativeBalanceException**.

Ngược lại, trừ tiền và in "Rút tiền thành công. Số dư còn lại: x".

3. Trong main:

Nhập: **username, password, số dư ban đầu, số tiền cần rút**

Tạo đối tượng **BankAccount**, gọi phương thức **withdraw**.

Sử dụng:

- try/catch để bắt WeakPasswordException và NegativeBalanceException.
- finally để in "Giao dịch kết thúc."

Output:

Trường hợp thành công

Nhập username: hoidanit

Nhập password: 123456

Nhập số dư: 5000

Nhập số tiền cần rút: 3000

Rút tiền thành công. Số dư còn lại: 2000.0

Giao dịch kết thúc.

Trường hợp password quá ngắn

Nhập username: hoidanit

Nhập password: 123

Nhập số dư: 5000

Nhập số tiền cần rút: 3000

WeakPasswordException: Mật khẩu phải có ít nhất 6 ký tự.

Giao dịch kết thúc.

Trường hợp rút quá số dư:

Nhập username: hoidanit

Nhập password: 123456

Nhập số dư: 2000

Nhập số tiền cần rút: 3000

NegativeBalanceException: Không đủ số dư để rút tiền.

Giao dịch kết thúc.

#105. Chữa Bài Tập Lab 08

//todo

@hoidanit

Chapter 11: Collection Framework

Giới thiệu Collection Framework với các interface và class chính: List, Set, Map, Queue.

Bao gồm cách duyệt, sắp xếp và sử dụng collection đồng bộ trong Java.

#106. Tổng quan về chapter 11

//todo

#107. Big O là gì (Extra)

1. Big O là gì?

Big O (Ký hiệu O) là cách mô tả tốc độ tăng trưởng thời gian chạy hoặc sử dụng bộ nhớ của thuật toán khi kích thước dữ liệu (n) thay đổi.

Nó không cho biết thời gian chạy cụ thể, mà cho biết xu hướng khi dữ liệu lớn lên.

Big O đo lường gì:

- Thời gian chạy (Time Complexity): tốc độ chạy khi dữ liệu tăng.
- Không gian sử dụng (Space Complexity): bộ nhớ cần thiết khi dữ liệu tăng.

Tại sao Big O quan trọng:

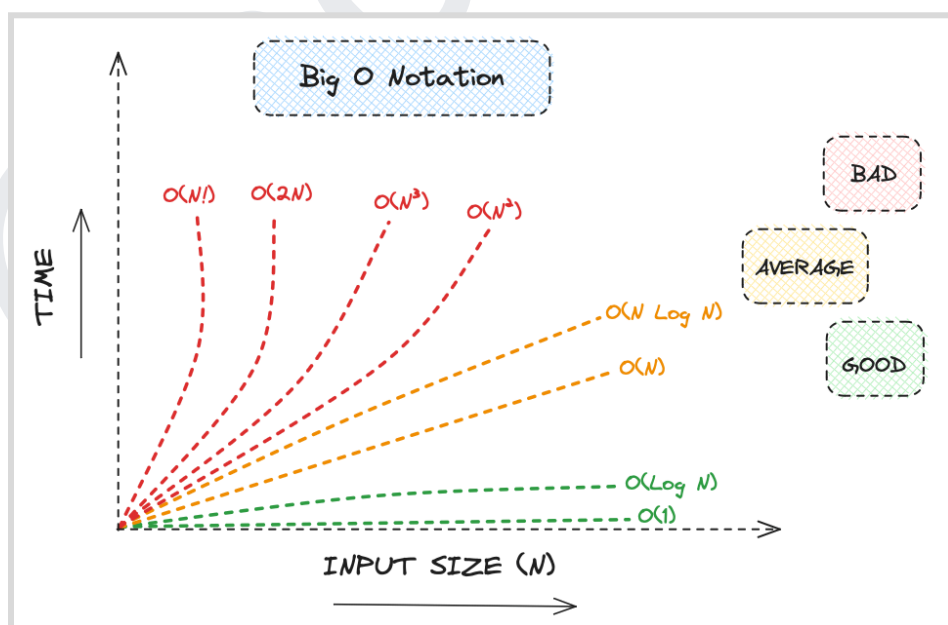
- Giúp bạn so sánh các thuật toán: thuật toán nào tối ưu hơn khi xử lý dữ liệu lớn.
- Trong phỏng vấn, Big O là phần quan trọng nhất để đánh giá tư duy giải thuật của bạn.

Ví dụ:

- Nếu thuật toán của bạn chạy trong **$O(n)$** , nghĩa là:
Nếu dữ liệu gấp đôi, thời gian chạy cũng gấp đôi.
- Nếu thuật toán chạy trong **$O(1)$** :
Dữ liệu gấp đôi, gấp 1000, thời gian chạy vẫn không đổi.

2. Các mức Big O thường gặp

Big O	Tên gọi	Ý nghĩa thực tế	Ví dụ
$O(1)$	Hằng số (Constant time)	Thời gian chạy không đổi dù dữ liệu lớn hay nhỏ	Truy cập <code>arr[i]</code> trong mảng
$O(\log n)$	Logarithmic time	Tăng chậm khi dữ liệu tăng	Tìm kiếm nhị phân trong mảng đã sắp xếp
$O(n)$	Tuyến tính (Linear time)	Tăng tỷ lệ thuận với dữ liệu	Duyệt qua tất cả phần tử (linear search)
$O(n \log n)$	Linearithmic time	Lớn hơn $O(n)$ nhưng nhỏ hơn $O(n^2)$	Thuật toán sắp xếp nhanh (quick sort), merge sort
$O(n^2)$	Bậc hai (Quadratic time)	Tăng nhanh khi dữ liệu tăng, gấp đôi dữ liệu -> gấp 4 thời gian chạy	Sắp xếp chọn (selection sort), bubble sort
$O(2^n)$	Hàm mũ (Exponential time)	Tăng cực nhanh, không khả thi với dữ liệu lớn	Thuật toán quay lui (backtracking) không tối ưu
$O(n!)$	Giai thừa (Factorial time)	Tăng rất nhanh, chỉ chạy được với dữ liệu rất nhỏ	Giải bài toán hoán vị toàn bộ (permutation)



#108. Tổng quan về Collection Framework

1. Collection Framework là gì?

Collection Framework là một **hệ thống các interface và class** trong Java để **lưu trữ và quản lý nhóm dữ liệu một cách hiệu quả**.

Trước đây, nếu không có Collection Framework, ta phải tự tạo array, linked list, stack... gây tốn thời gian và dễ lỗi.

Collection Framework cung cấp sẵn:

- **Các cấu trúc dữ liệu** (data structures) như list, set, map.
- **Các thuật toán** (algorithms) để sắp xếp, tìm kiếm.
- **Các interface chuẩn** để code linh hoạt, dễ mở rộng.

✓ Ví dụ thực tế:

- Danh sách học sinh trong lớp → dùng List.
- Danh sách mã nhân viên (không trùng) → dùng Set.
- Danh bạ điện thoại (số – tên) → dùng Map.

2. Các interface chính

Interface	Đặc điểm	Khi nào dùng	Ví dụ class phổ biến
List	Lưu theo thứ tự Cho phép phần tử trùng lặp.	Khi cần lưu danh sách có thứ tự, cho phép trùng	ArrayList, LinkedList
Set	Không cho phép phần tử trùng lặp Không đảm bảo thứ tự (HashSet) hoặc có thứ tự (TreeSet)	Khi cần dữ liệu unique (không trùng)	HashSet, TreeSet
Map	Lưu key-value pair Key không trùng lặp.	Khi cần tra cứu theo key	HashMap, TreeMap
Queue	Hàng đợi, thường xử lý FIFO	Khi cần xử lý tuần tự (vd: hệ thống xếp hàng)	LinkedList, PriorityQueue

#109. List và ArrayList

1.Interface List là gì?

List là một interface trong Collection Framework, đại diện cho **danh sách các phần tử có thứ tự**, cho phép trùng lặp.

Mỗi phần tử trong List có một index (chỉ số), bắt đầu từ 0.

Các class implement List hay dùng: **ArrayList**, LinkedList.

2.ArrayList

ArrayList là class phổ biến nhất implement List.

Cấu trúc: mảng động, tự mở rộng khi thêm phần tử mới.

Ưu điểm:

- Truy xuất phần tử theo index nhanh ($O(1)$).
- Dễ sử dụng, phù hợp khi chủ yếu đọc dữ liệu.

Nhược điểm:

Thêm hoặc xóa phần tử ở giữa danh sách chậm ($O(n)$) vì cần dịch chuyển các phần tử sau.

Ví dụ về cách khai báo ArrayList:

```
ArrayList<String> fruits = new ArrayList<>();
```

```
//todo
```

#110. LinkedList (Extra)

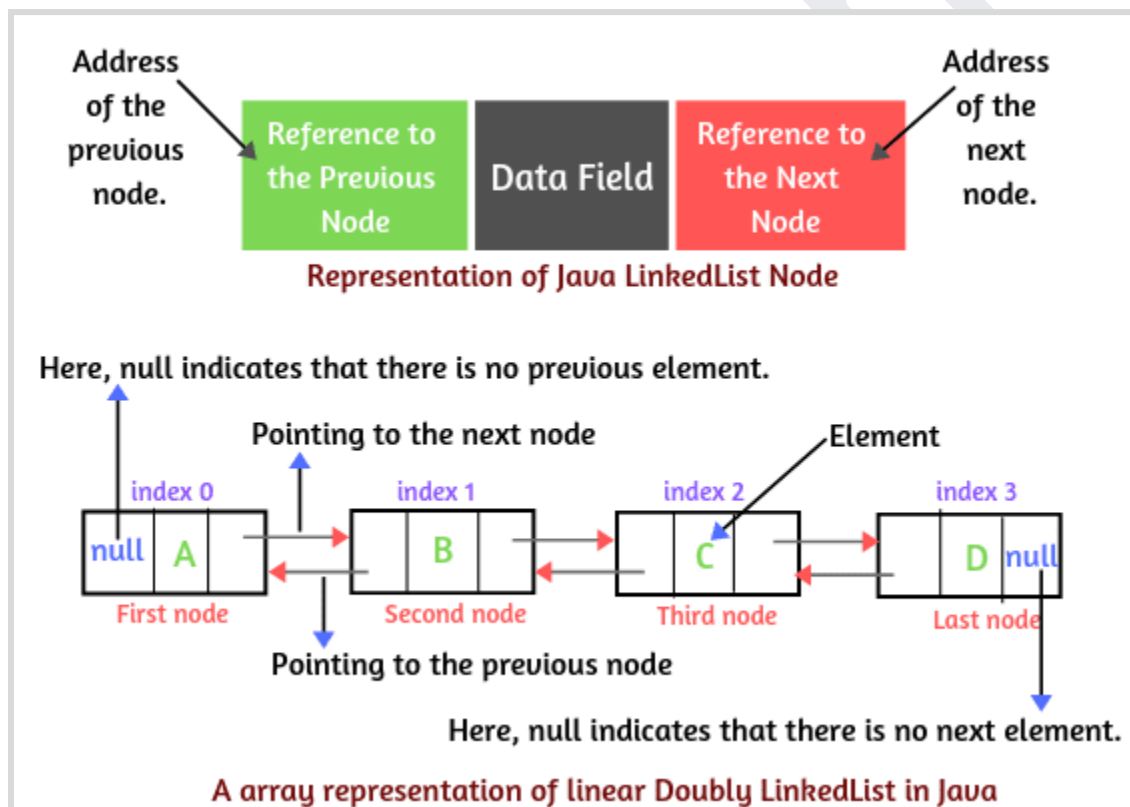
//todo: <https://www.baeldung.com/java-diamond-operator>

1. LinkedList là gì?

LinkedList là class implements interface List trong Java.

Cấu trúc dữ liệu: danh sách liên kết đôi (doubly linked list). Mỗi phần tử (node) chứa dữ liệu và 2 con trỏ:

- **previous** (trở tới node trước)
- **next** (trở tới node sau)



Đặc điểm:

- Thêm hoặc xóa phần tử ở đầu/cuối list nhanh ($O(1)$).
- Truy xuất phần tử theo index chậm ($O(n)$) vì phải duyệt từ đầu.

#111. Bài Tập về ArrayList

//todo: Phân biệt List và ArrayList

Yêu cầu:

Tạo class **Student** với:

- **Thuộc tính:**
String name
int age
- Constructor để khởi tạo tất cả thuộc tính.
- Phương thức **toString()** để in ra thông tin theo định dạng:
"Name: <name>, Age: <age>"

Trong phương thức main, Khai báo ArrayList<Student> students:

- Thêm 3 sinh viên với thông tin tùy chọn.
Duyệt qua danh sách và in ra từng sinh viên.
- Thêm 1 sinh viên mới vào index=1.
Duyệt qua danh sách và in ra từng sinh viên.
- Xóa sinh viên ở index=0.
Duyệt qua danh sách và in ra từng sinh viên.

Output:

Danh sách ban đầu:

Name: Alice, Age: 20

Name: Bob, Age: 22

Name: Charlie, Age: 21

Sau khi thêm sinh viên vào index=1:

Name: Alice, Age: 20

Name: David, Age: 23

Name: Bob, Age: 22

Name: Charlie, Age: 21

Sau khi xóa sinh viên index=0:

Name: David, Age: 23

Name: Bob, Age: 22

Name: Charlie, Age: 21

#112. Set, HashSet và TreeSet

1.Interface Set là gì?

Định nghĩa:

- Set là một interface trong Collection Framework.
- Dùng để lưu trữ **tập hợp các phần tử không trùng lặp**.

Đặc điểm:

- Không cho phép các phần tử giống hệt nhau cùng tồn tại.
- **Không có index để truy xuất phần tử**, khác với List.

Ví dụ:

- Danh sách email(không trùng).
- Danh sách số điện thoại trong công ty.

2.HashSet là gì?

- Là class implements interface Set.
- Dựa trên **hash table** để lưu trữ.
- **Không lưu trữ phần tử trùng lặp**.
- **Không đảm bảo thứ tự** các phần tử khi in ra.

Khi nào nên dùng HashSet?

Khi chỉ cần kiểm tra dữ liệu unique và **không quan tâm** thứ tự lưu trữ.

//Ví dụ: **HashSet**<String> names = new **HashSet**<>();

3.TreeSet là gì?

- Là class implements interface Set.
- Dựa trên cây nhị phân tìm kiếm (Red-Black Tree : cây đỏ/đen ; cây nhị phân tự cân bằng).
- **Không chứa phần tử trùng lặp.**
- **Tự động sắp xếp** các phần tử theo thứ tự tăng dần.

Khi nào nên dùng TreeSet?

Khi cần dữ liệu unique và phải được sắp xếp theo thứ tự tự nhiên hoặc Comparator.

Ví dụ: `TreeSet<Integer> numbers = new TreeSet<>();`

So sánh HashSet và TreeSet:

Tiêu chí	HashSet	TreeSet
Cấu trúc lưu trữ	Hash Table	Red-Black Tree
Thứ tự phần tử	Không đảm bảo	Tự động sắp xếp tăng dần
Cho phép null	Cho phép 1 phần tử null	Không cho phép null
Hiệu suất (thêm/xóa/tìm)	Nhanh hơn, $O(1)$	Chậm hơn, $O(\log n)$
Khi nào dùng	Khi chỉ cần unique, không quan tâm thứ tự	Khi cần unique + sắp xếp

#113. Quá trình thêm phần tử vào HashSet/HashMap (Extra)

Tài liệu: https://www.youtube.com/watch?v=KyUTuwz_b7Q

1. Hash Table là gì?

Là cấu trúc dữ liệu lưu trữ key (hoặc key-value) dựa trên hàm băm (hash function).

Hàm băm nhận đầu vào (key), trả về một số nguyên → dùng để xác định vị trí (bucket) trong table.

2. Quá trình thêm phần tử vào HashSet/HashMap

Bước 1: Tính hashCode()

Khi bạn thêm một phần tử (vd: object Student) vào HashSet: **students.add(student);**
Java gọi student.**hashCode()** để tính ra giá trị hash.

Bước 2: Xác định bucket

Hash table sử dụng giá trị hashCode để xác định bucket (vị trí lưu).

- Nếu bucket trống (chưa có phần tử nào), Java thêm object vào bucket đó. Xong.

Bước 3: Nếu bucket đã có object khác (collision)

Nhiều key khác nhau có thể cho cùng một bucket (vì hashCode trùng hoặc table size nhỏ). Đây gọi là collision (xung đột).

Java sẽ duyệt qua các phần tử trong bucket đó (thường là linked list hoặc tree từ Java 8+).

Bước 4: Gọi `equals()` để so sánh

Với mỗi object trong bucket, Java gọi:

`student.equals(existingObject)`

Nếu `equals()` trả về `true`, Java coi object mới trùng lặp với object cũ

→ KHÔNG thêm vào HashSet (Set không cho phép trùng).

Nếu `equals()` trả về `false` (với tất cả object trong bucket), Java thêm object mới vào bucket (dùng chaining).

3. Tại sao cần cả `hashCode()` và `equals()`?

<code>hashCode()</code>	<code>equals()</code>
Xác định bucket	Xác định trùng lặp trong bucket
Nhanh, chỉ tính toán số nguyên	So sánh giá trị chi tiết từng field

- Nếu chỉ dựa vào `equals()` mà không có `hashCode()`, mọi phần tử sẽ bị đưa vào cùng 1 bucket → hiệu suất $O(n)$.
- Nếu chỉ dựa vào `hashCode()` mà không `equals()`, 2 object khác field nhưng trùng `hashCode` vẫn bị coi là bằng nhau → sai logic.

#114. Bài Tập HashSet

Yêu cầu:

Tạo class **Product**

- Thuộc tính:
 int **id**
 String **name**
 double **price**
- Constructor khởi tạo đầy đủ, getter, setter
- Override **toString()** in theo định dạng
- Override **equals()** và **hashCode()**

Trong class main:

Khai báo HashSet<Product> products.

Thêm các sản phẩm:

(1, "Laptop", 1500.0)

(2, "Mouse", 20.0)

(3, "Keyboard", 45.0)

(1, "Laptop Pro", 2000.0) // Trùng id với Laptop

(1, "Laptop", 1500.0) // Trùng id tất cả thuộc tính

In ra danh sách products:

- Xóa sản phẩm có id=3 khỏi set.

#115. Map, HashMap và TreeMap

1. Map là gì?

- Map là interface lưu trữ dữ liệu dưới dạng **cặp key-value (khóa – giá trị)**.
- **Key không trùng lặp**, mỗi key ánh xạ tới 1 value.

Ví dụ thực tế:

Tình huống	Key	Value
Danh bạ điện thoại	Số điện thoại	Tên người
Danh sách sinh viên	Mã sinh viên	Tên sinh viên
Giỏ hàng	Mã sản phẩm	Số lượng

2. So sánh HashMap và TreeMap

Tiêu chí	HashMap	TreeMap
Cấu trúc	Hash Table	Red-Black Tree
Thứ tự phần tử	Không đảm bảo	Tự động sắp xếp theo key
Cho phép null key?	✓ Có	✗ Không
Tốc độ thao tác	Nhanh hơn ($O(1)$ trung bình)	Chậm hơn ($O(\log n)$)
Khi nào dùng?	Khi chỉ cần lưu key-value, không cần sắp xếp	Khi cần dữ liệu key-value được sắp xếp

Khai báo:

```
HashMap<Integer, String> students = new HashMap<>();
```

```
TreeMap<Integer, String> students = new TreeMap<>();
```

Duyệt:

```
for (Map.Entry<Integer, String> entry : students.entrySet()) {  
    System.out.println("ID: " + entry.getKey() + ", Name: " + entry.getValue());  
}
```

#116. Iterator và ListIterator

Mục tiêu:

- Hiểu **Iterator** và **ListIterator** là gì, cách hoạt động.
- Biết vì sao cần Iterator, khác gì với for và for-each.
- Thực hành duyệt, xóa, thêm phần tử trong Collection an toàn.

1. Vấn đề đặt ra khi sử dụng for/foreach

```
ArrayList<String> languages = new ArrayList<>();  
languages.add("Java");  
languages.add("C#");  
languages.add("PHP");  
languages.add("C++");  
languages.add("JavaScript");
```

```
// throw ConcurrentModificationException  
for (String language : languages) {  
    if (language.equals("C#")) {  
        languages.remove(language);  
    }  
}
```

2.Iterator vs ListIterator

Tiêu chí	Iterator	ListIterator
Thuộc interface	Iterator	ListIterator (extends Iterator)
Áp dụng cho	Tất cả Collection: List, Set, Queue	Chỉ dùng cho List
Duyệt forward (tiến tới)	✓ Có	✓ Có
Duyệt backward (lùi lại)	✗ Không	✓ Có (hasPrevious(), previous())
Thêm phần tử khi duyệt	✗ Không	✓ Có (add())
Sửa phần tử khi duyệt	✗ Không	✓ Có (set())
Xóa phần tử khi duyệt	✓ Có (remove())	✓ Có (remove())
Trả về index	✗ Không	✓ Có (nextIndex(), previousIndex())
Tính năng đặc biệt	Đơn giản, chỉ duyệt tiến	Đầy đủ: duyệt 2 chiều, thêm, sửa, xóa
Độ phức tạp API	Đơn giản hơn	Phức tạp hơn

Ví dụ về **Iterator**:

```
ArrayList<String> names = new ArrayList<>();
names.add("Charlie");
```

```
Iterator<String> it = names.iterator();
```

```
while (it.hasNext()) {
    String name = it.next();
    if (name.equals("Bob")) {
        it.remove(); // Xóa an toàn
    }
}
```

Ví dụ về ListIterator:

```
public class ListIteratorDemo {  
    public static void main(String[] args) {  
        ArrayList<Integer> nums = new ArrayList<>();  
        nums.add(10);  
  
        ListIterator<Integer> li = nums.listIterator();  
  
        System.out.println("Duyệt xuôi:");  
        while (li.hasNext()) {  
            System.out.println(li.next());  
        }  
  
        System.out.println("Duyệt ngược:");  
        while (li.hasPrevious()) {  
            System.out.println(li.previous());  
        }  
  
        // Thêm phần tử 25 vào sau 20  
        li = nums.listIterator();  
        while (li.hasNext()) {  
            if (li.next() == 20) {  
                li.add(25); // Thêm sau 20  
            }  
        }  
  
        System.out.println("Sau khi thêm 25: " + nums);  
    }  
}
```

#117. Bài Tập Lab 09

Yêu cầu: Tạo class Member

1. Thuộc tính:

int **id**

String **name**

int **age**

Constructor, toString(), getter, setter

2. Implement Comparable cho Member

Gợi ý cách sắp xếp [tại đây](#)

- Sắp xếp theo tuổi tăng dần.
- Nếu tuổi bằng nhau, sắp xếp theo tên (alphabet).

3. Quản lý danh sách thành viên

- Danh sách đăng ký (**ArrayList**)
Khai báo **ArrayList**<Member> members.
Thêm 5 thành viên vào danh sách.
In danh sách sau khi thêm.
- Danh sách ID thành viên duy nhất (**HashSet**) => cần viết hashCode/Equal
Khai báo **HashSet**<Integer> memberIDs.
Thêm id của các thành viên vào HashSet.
Thêm id bị trùng và kiểm tra có thêm được không.
In danh sách ID.
- Danh sách thành viên xếp hạng theo tuổi (**TreeSet**)
Khai báo **TreeSet**<Member> rankedMembers.
Thêm các thành viên trên vào TreeSet.
In danh sách rankedMembers.

4. Quản lý điểm thưởng hoạt động (Map)

a. **HashMap** – Điểm thưởng

Khai báo `HashMap<Integer, Integer> pointMap` lưu id → điểm thưởng.

Thêm điểm thưởng cho 3 thành viên bất kỳ.

Cộng thêm 10 điểm cho member id=1 (nếu tồn tại).

In danh sách điểm thưởng.

b. **TreeMap** – ID → Name

Khai báo `TreeMap<Integer, String> idNameMap`.

Thêm toàn bộ thành viên vào `TreeMap`.

In ra `TreeMap`, quan sát thứ tự key.

#118. Chữa Bài Tập Lab 09

//todo

Chapter 12: Generics trong Java

Giới thiệu Generics – cách viết class và method tổng quát

#119. Tổng quan về chapter 12

//todo

#120. Generics là gì ?

1. Khái niệm

Generics cho phép viết code tổng quát mà vẫn đảm bảo **an toàn kiểu dữ liệu** (type safety).

Bạn có thể định nghĩa class, method hoạt động với nhiều kiểu dữ liệu khác nhau mà không cần ép kiểu (casting) thủ công.

Trước khi có Generics:

```
List list = new ArrayList();  
list.add("Hello");  
String s = (String) list.get(0); // Phải cast về String
```

Sau khi có Generics:

```
List<String> list = new ArrayList<>();  
list.add("Hello");  
String s = list.get(0); // Không cần cast, an toàn kiểu dữ liệu
```

2. Tại sao cần Generics?

Generics giải quyết vấn đề:

- Tránh lỗi runtime do cast sai kiểu.
- Giúp code rõ ràng, dễ đọc, dễ bảo trì.
- Tái sử dụng class và method cho nhiều kiểu dữ liệu khác nhau.

#121. Generic Class và Generic Method

1. Generic Class

Generic class là class được định nghĩa với tham số kiểu (type parameter), giúp class làm việc với nhiều kiểu dữ liệu khác nhau mà vẫn đảm bảo an toàn kiểu dữ liệu (type safety).

Tại sao cần Generic Class?

Trước Generics, bạn phải dùng Object cho mọi kiểu dữ liệu → cần casting thủ công khi lấy ra, dễ gây lỗi runtime.

Với Generic Class, compiler sẽ kiểm tra kiểu dữ liệu ngay từ compile time.

Ví dụ:

```
class InventoryItem<T> {  
    private T id;  
    private String name;  
  
    public InventoryItem(T id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public T getId() { return id; }  
    public String getName() { return name; }  
}
```

```
InventoryItem<Integer> item1 = new InventoryItem<>(101, "Laptop");  
InventoryItem<String> item2 = new InventoryItem<>("SKU555", "iPhone");
```

2. Generic Method

Generic method là method khai báo tham số kiểu riêng trong chính method đó.

Nó hoạt động độc lập, không phụ thuộc class có generic hay không.

Ví dụ:

```
public static <T> void printArray(T[] arr) {  
    for (T e : arr) {  
        System.out.print(e + " ");  
    }  
    System.out.println();  
}  
  
printArray(names); // Alice Bob  
printArray(nums); // 1 2 3
```

3. Lưu ý

- Có thể khai báo nhiều type parameter trong một Generic Class hoặc Generic Method, chỉ cần phân tách bằng dấu phẩy.
- T chỉ là quy ước đặt tên phổ biến, nhưng bạn có thể đặt bất kỳ tên nào. Tuy nhiên, Java community có các quy ước đặt tên tham số kiểu để code rõ ràng hơn:

Ký hiệu	Ý nghĩa thường dùng
T	Type
E	Element (thường dùng trong Collection)
K	Key
V	Value
N	Number
S, U, V	Type thứ 2, 3, 4 nếu cần

#122. Bài Tập về Generics

Yêu cầu:

Tạo class generic **MyTransaction**<ID, AMOUNT, STATUS>

Thuộc tính:

private ID transactionId; // Mã giao dịch, có thể là Integer hoặc String

private AMOUNT amount; // Số tiền, có thể là Double hoặc Integer.

private STATUS status; // Trạng thái, có thể là String ("SUCCESS", "FAILED") hoặc Enum TransactionStatus.

//tạo mới đối tượng, in ra thông tin của đối tượng đấy

#123. Unbounded Wildcard (<?>) - Wildcard không giới hạn

1. <?> là gì?

<?> là unbounded wildcard (wildcard không ràng buộc/không giới hạn).

Đại diện cho “một kiểu dữ liệu bất kỳ”.

Dùng khi bạn muốn **method** chấp nhận mọi loại data type, nhưng không quan tâm kiểu cụ thể bên trong.

Ví dụ:

```
public static void printList(List<?> list) {  
    for (Object item : list) {  
        System.out.println(item);  
    }  
}
```

2. So sánh Wildcard <?> với Generic T

Tiêu chí	Wildcard <?>	Generic T (type parameter)
Ý nghĩa	Đại diện cho một kiểu dữ liệu bất kỳ (unknown type)	Đại diện cho một kiểu dữ liệu cụ thể , do caller quyết định
Khai báo method	Không cần khai báo <T> trước method	Phải khai báo <T> trước method
Khai báo class	✗ Không thể dùng <?> để định nghĩa class generic	✓ Dùng để định nghĩa class generic
Add dữ liệu	✗ Không thể add (trừ null)	✓ Có thể add dữ liệu kiểu T
Update dữ liệu	✗ Không thể set dữ liệu	✓ Có thể set dữ liệu kiểu T
Delete dữ liệu	✓ Có thể remove phần tử	✓ Có thể remove phần tử
Get dữ liệu	✓ Trả về Object (cần cast nếu muốn kiểu cụ thể)	✓ Trả về T, không cần cast
Dùng khi	Khi method chỉ cần đọc hoặc xóa dữ liệu, không cần biết kiểu cụ thể	Khi class/method làm việc với một kiểu dữ liệu cụ thể, cần add, update, hoặc return kiểu đó

Ví dụ minh họa:

```
//todo
```

@hoidanit

#124. Upper Bounded Wildcard (<? extends T>) - Wildcard giới hạn trên

1. Upper Bounded Wildcard là gì?

Khai báo: **<? extends T>**

Nghĩa là kiểu dữ liệu không xác định, nhưng bị giới hạn trên bởi T.

Chấp nhận T hoặc bất kỳ subclass (lớp con) nào của T.

Cách dùng: Thường được sử dụng khi bạn **muốn đọc dữ liệu** từ một collection.

Nó đảm bảo rằng các đối tượng trong collection có thể được coi là kiểu T hoặc một kiểu con của T.

Bạn **không thể thêm đối tượng** vào một collection được khai báo với <? extends T> (trừ null)

Ví dụ:

```
public static void printNumbers(List<? extends Number> list) {  
    for (Number n : list) {  
        System.out.println(n);  
    }  
}
```

<? extends Number> chấp nhận:

List<Integer>

List<Double>

List<Float>

#125. Lower Bounded Wildcard (<? super T>) - Wildcard giới hạn dưới

1.<? super T> là gì?

Khai báo: <? super T>

Là Wildcard giới hạn dưới (Lower Bounded Wildcard).

Đại diện cho T hoặc bất kỳ superclass (lớp cha) nào của T.

Cách dùng: Thường được sử dụng khi bạn **muốn thêm dữ liệu vào một collection**.

Nó đảm bảo rằng bạn có thể thêm các đối tượng **kiểu T hoặc các lớp con của T** vào collection đó.

Ví dụ:

```
public static void addIntegers(List<? super Integer> list) {  
    list.add(10);  
    list.add(20);  
}
```

<? super Integer> có thể là:

List<Integer>

List<Number>

List<Object>

//lưu ý:

Khi get phần tử từ List<? super Integer>, compiler chỉ đảm bảo nó là Object, không chắc chắn là Integer.

Object item = list.get(0); //  OK, nhưng chỉ là Object

// Integer num = list.get(0); //  Compile error

#126. Type Erasure (Extra)

1. Khái niệm Type Erasure

Type Erasure (Xóa kiểu) là cơ chế xử lý Generics của Java **khi biên dịch (compile time)**.

Trong Java, **Generics chỉ tồn tại ở compile time** để kiểm tra kiểu dữ liệu (type safety). **Khi chạy (runtime), mọi thông tin về Generics sẽ bị xóa bỏ**, và JVM chỉ thấy kiểu raw type (Object hoặc upper bound).

Code Generic trước khi biên dịch:

```
List<String> list = new ArrayList<>();  
list.add("Hello");  
String s = list.get(0);
```

Sau khi biên dịch (Type Erasure):

```
List list = new ArrayList();  
list.add("Hello");  
String s = (String) list.get(0);
```

Hệ quả	Giải thích
✓ Không thể dùng primitive type với Generics	Vì sau erase, Generics thành Object, mà Object không chứa kiểu primitive (int, double). Phải dùng wrapper class (Integer, Double)
✓ Không thể tạo object kiểu T	Ví dụ: new T() lỗi compile, vì JVM không biết T là gì tại runtime
✓ Không thể tạo array kiểu T[]	Ví dụ: T[] arr = new T[10]; lỗi compile, vì JVM không biết T là gì

#127. Bài Tập Lab 10

Yêu cầu:

Tạo class generic **InventoryItem<T>**:

Thuộc tính:

private **T id**; // Mã sản phẩm có thể là Integer (ID số) hoặc String (SKU).

private **String name**;

private **double price**;

Constructor khởi tạo toàn bộ thuộc tính, getter/setter

Tạo class **InventoryUtils** chứa generic method:

public static <T> void **printItemInfo**(InventoryItem<T> item)

In thông tin sản phẩm theo format:

ID: <id>, Name: <name>, Price: <price>

public static <T> double **calculateTotalPrice**(List<InventoryItem<T>> items)

Trả về tổng giá tất cả sản phẩm trong list: Total price: <value>

Trong class Main, thực hiện:

Tạo list sản phẩm gồm:

InventoryItem<Integer> (vd: ID số).

InventoryItem<String> (vd: SKU mã vạch).

Dùng **printItemInfo** để in thông tin từng sản phẩm.

Gọi **calculateTotalPrice** và in ra tổng tiền

Ví dụ về output:

ID: 1001, Name: Laptop Dell, Price: 1500.0

ID: SKU12345, Name: iPhone 15, Price: 1200.0

Total price: 2700.0

#128. Chữa Bài Tập Lab 10

//todo

Chapter 13: Java I/O (Input/Output)

Xử lý Đọc/ghi file với Java

#129. Tổng quan về chapter 13

//todo

#130. Phân Loại Đọc/Ghi File ?

Khi đọc/ghi file trong Java, có 2 loại chính:

- **Character** Streams (Luồng ký tự): đã được giải mã thành ký tự Unicode, ví dụ file văn bản/ký tự: .txt, .csv, .java, .html...
- **Byte** Streams (Luồng nhị phân): xử lý dữ liệu dưới dạng byte thô, dùng cho mọi loại file (image, audio, video, file đã biên dịch .bin, .dat...)

1. Luồng Ký tự (Character Streams)

Các class cần tập trung:

Class	Mục đích sử dụng
FileReader & FileWriter	Đọc/Ghi file text cơ bản theo từng ký tự. Học để có nền tảng
PrintWriter	Tương tự FileWriter, cung cấp theo các method để "format" data trước khi lưu
BufferedReader & BufferedWriter	Luôn dùng khi đọc/ghi file text thực tế vì: ✓ Đọc theo dòng (readLine). ✓ Buffer tăng tốc độ.

Ưu tiên:

- Đọc file với **BufferedReader**
- Ghi file với **BufferedWriter + PrintWriter**

2. Luồng nhị phân (Byte Streams)

Class	Mục đích sử dụng
FileInputStream & FileOutputStream	Đọc/Ghi file nhị phân cơ bản theo từng byte
BufferedInputStream & BufferedOutputStream	Sử dụng buffer, giúp tăng tốc độ đọc/ghi file nhị phân lớn

#131. Đọc/Ghi file với FileReader/Writer/PrintWriter

1. FileReader là gì?

FileReader là class dùng để đọc file văn bản (text file), **theo từng ký tự** (character).

Phù hợp khi bạn cần đọc toàn bộ file, duyệt từng ký tự riêng lẻ.

Cú pháp:

```
FileReader fr = new FileReader("input.txt"); // Mở file
int c;
while ((c = fr.read( )) != -1) { // Đọc từng ký tự, trả về int
    System.out.print((char) c); // Ép int về char để hiển thị
}
```

- **fr.read()** đọc 1 ký tự, trả về mã ASCII hoặc UTF-16 (Unicode) dạng int.
- **Trả về -1** khi tới cuối file (EOF).

2. FileWriter là gì?

FileWriter là class dùng để ghi dữ liệu text vào file, từng ký tự hoặc chuỗi.

Cú pháp:

```
FileWriter fw = new FileWriter("output.txt"); // Tạo file output.txt
fw.write("Hello, world!\n");
fw.write("This is FileWriter example.\n");
```

- **fw.write()** ghi dữ liệu vào file.
- File sẽ được tạo mới nếu chưa tồn tại, hoặc ghi đè nếu đã tồn tại.

3. **PrintWriter** – Ghi file văn bản dễ dàng hơn

Hỗ trợ các phương thức quen thuộc như `print()`, `println()`, `printf()` giống `System.out`

Cú pháp:

```
PrintWriter pw = new PrintWriter("output2.txt");  
    pw.println("Hello, PrintWriter!");  
    pw.printf("Number: %d, String: %s\n", 10, "Java");
```

#132. Đọc/Ghi file với **BufferedReader/BufferedWriter**

1. Buffer là gì?

Buffer (bộ đệm) là vùng nhớ tạm nằm giữa chương trình và thiết bị IO (ổ cứng, mạng).

Khi đọc file: thay vì đọc từng ký tự hoặc byte từ ổ cứng

→ chương trình đọc một khối lớn dữ liệu (chunk) vào buffer, sau đó lấy dần từng phần trong RAM → nhanh hơn.

Khi ghi file: thay vì ghi từng ký tự xuống ổ cứng (rất tốn thời gian IO)

→ chương trình ghi dữ liệu vào buffer trước, khi buffer đầy hoặc đóng stream

→ ghi một lần xuống ổ cứng → tiết kiệm thời gian.

2. **BufferedReader** là gì?

BufferedReader là class dùng để:

- Đọc file text theo từng dòng (`readLine()`).
- Đọc nhanh hơn **FileReader** nhờ buffer nội bộ.

Cú pháp:

```
FileReader fr = new FileReader("input.txt");  
BufferedReader br = new BufferedReader(fr);  
  
String line;  
while ((line = br.readLine()) != null) { // đọc từng dòng  
    System.out.println(line);  
}
```

3. **BufferedWriter** là gì?

BufferedWriter là class dùng để:

- Ghi file text nhanh hơn **FileWriter** nhờ buffer.
- Hỗ trợ `newLine()` để xuống dòng.

Cú pháp:

```
FileWriter fw = new FileWriter("output.txt");  
BufferedWriter bw = new BufferedWriter(fw);
```

```
bw.write("Hello, BufferedWriter!");  
bw.newLine(); // xuống dòng  
bw.write("This is fast writing example.");
```

#133. Sử dụng java.nio.file.Files (Character Stream)

1. java.nio.file.Files là gì?

Files là class utility trong package java.nio.file, cung cấp các phương thức static để thao tác file và thư mục như: đọc file, ghi file, copy, move, delete file.

Ưu điểm:

- ✓ Code ngắn gọn, dễ dùng.
- ✓ Không cần tạo nhiều object như FileReader, BufferedReader.
- ✓ Hỗ trợ đọc ghi file text nhanh chóng cho **file nhỏ – vừa**.

2. Đọc file text bằng Files

```
List<String> lines = Files.readAllLines(Paths.get("input.txt"));
    for (String line : lines) {
        System.out.println(line);
    }
```

3. Ghi file text bằng Files

```
List<String> lines = Arrays.asList(
    "Hello, java.nio.file.Files!",
    "This is write() example."
);
Files.write(Paths.get("output.txt"), lines);
```

#134. Bài Tập Đọc/Ghi File (Character Streams)

Yêu cầu:

Cho file “**students.txt**” chứa dữ liệu sinh viên, mỗi dòng có định dạng:

Name,Score

Ví dụ định dạng của students.txt:

hoidanit,8.5

Nam,6.0

An,9.2

Eric,7.5

Viết chương trình:

- ✓ **Bước 1:** Đọc file “students.txt” sử dụng **BufferedReader**.
- ✓ **Bước 2:** Với mỗi dòng, tách thành **tên sinh viên** và **điểm** (double), lưu vào danh sách object **Student**.
- ✓ **Bước 3:** Tính điểm trung bình của lớp.
- ✓ **Bước 4:** Xuất báo cáo vào file “**report.txt**” sử dụng **PrintWriter**, với format sau:

===== STUDENT REPORT =====

Name	Score	Pass/Fail
------	-------	-----------

hoidanit	8.50	PASS
----------	------	------

Nam	6.00	PASS
-----	------	------

An	9.20	PASS
----	------	------

Eric	7.50	PASS
------	------	------

Average Score: 7.80

=====

Quy định Pass/Fail:

Điểm $\geq 5.0 \rightarrow$ PASS

Điểm $< 5.0 \rightarrow$ FAIL

✅ Bước 5. In ra console dòng thông báo:
Report generated successfully to report.txt

#135. Đọc/Ghi file với FileInputStream & FileOutputStream

1. FileInputStream

Ví dụ: Đọc file ảnh (input.jpg) và in số byte

Download file hình ảnh trong video [tại đây](#)

```
FileInputStream fis = new FileInputStream(filePath)
```

```
int sizeInBytes = fis.available(); // available trả về số byte có thể đọc  
double sizeInMB = (double) sizeInBytes / (1024 * 1024);  
System.out.printf("Kích thước file: %.2f MB\n", sizeInMB);
```

2. FileOutputStream

Ví dụ: copy file

```
String inputPath = "input.jpg";  
String outputPath = "copy.jpg";
```

```
FileInputStream fis = new FileInputStream(inputPath);  
FileOutputStream fos = new FileOutputStream(outputPath)
```

```
int b;  
while ((b = fis.read()) != -1) {  
    fos.write(b);  
}
```

#136. Đọc/Ghi file với `BufferedInputStream` & `BufferedOutputStream`

//todo

#137. Sử dụng `java.nio.file.Files` (Byte Stream)

1. `Files.readAllBytes()`

```
byte[] data = Files.readAllBytes(Paths.get(filePath));
```

```
System.out.println("Đã đọc file: " + filePath);
```

```
System.out.println("Kích thước file: " + data.length + " bytes");
```

```
System.out.println("Giá trị byte đầu tiên: " + data[0]);
```

2. `Files.write()`

```
byte[] data = {10, 20, 30, 40, 50}; // mảng byte mẫu
```

```
Files.write(Paths.get(filePath), data);
```

```
System.out.println("Đã ghi file: " + filePath);
```

#138. Bài Tập Đọc/Ghi File (Byte Stream)

Yêu cầu: Viết chương trình Java thực hiện copy file

Bước 1: tạo đường dẫn file nguồn và file đích. Download file mẫu [tại đây](#)

String **sourcePath** = "input.mp3"; // đường dẫn file gốc

String **destPath** = "copy.mp3"; // đường dẫn file copy

Bước 2: Mở file nguồn để đọc

- Dùng **FileInputStream** để mở file nguồn.
- Bọc **FileInputStream** trong **BufferedInputStream** để tăng tốc độ đọc.

Bước 3: Mở file đích để ghi

- Dùng **FileOutputStream** để mở file đích.
- Bọc **FileOutputStream** trong **BufferedOutputStream** để tăng tốc độ ghi.

Bước 4: Copy dữ liệu từ file nguồn sang file đích

- Đọc từng byte từ **BufferedInputStream**.
- Ghi từng byte đó vào **BufferedOutputStream**.

Dùng vòng lặp:

```
int b;  
while ((b = bis.read()) != -1) {  
    bos.write(b);  
}
```

Bước 5: Đo thời gian chạy chương trình

Trước khi bắt đầu copy → lưu thời gian bắt đầu

Sau khi copy xong → lưu thời gian kết thúc

sử dụng: **System.currentTimeMillis()**;

In ra kết quả: Copy completed in XXX ms

#139. Serialization & Serializable Interface (Extra)

1. Khái niệm Serialization là gì?

Serialization là quá trình chuyển **object thành byte stream** (dãy byte).

Deserialization là quá trình ngược lại – chuyển byte stream về object gốc.

Serializable không bắt buộc để bạn viết code Java bình thường:

- Viết class, method, xử lý logic → không liên quan gì tới Serializable.
- Nếu chỉ chạy chương trình, tính toán, xử lý dữ liệu trong RAM → không cần.

Vậy khi nào mới cần Serializable?

- Khi bạn cần lưu object vào file hoặc database
- Khi cần gửi object qua network
- Khi sử dụng các framework Java (Spring Session / Redis cache...)

2. Serializable Interface là gì?

Serializable là một marker interface trong Java:

```
public interface Serializable { }
```

(Marker interface: không có method)

Tác dụng: Nếu class implements Serializable → JVM có thể:

- Biến object thành byte stream (serialize).
- Khôi phục object từ byte stream (deserialize).

#140. serialVersionUID & Warning khi extends Exception

1. serialVersionUID là gì?

serialVersionUID là một giá trị long, đại diện cho version (phiên bản) của class khi serialize.

Khai báo: `private static final long serialVersionUID = 1L;`

Câu hỏi:

- Có thể đặt giá trị khác 1L được không ?
- Các class khác nhau, cùng giá trị 1L được không ?

Nếu không khai báo serialVersionUID thì sao?

✗ JVM sẽ tự tạo một giá trị dựa trên class (fields, methods,...).

✗ Nếu class thay đổi (thêm/sửa/xoá field), JVM sẽ tạo serialVersionUID mới → file cũ deserialize bị lỗi.

✓ Best practice: Luôn khai báo serialVersionUID để giữ version class ổn định.

2. Warning khi extends Exception

//todo: fix warning tại video [#102](#)

Nguyên nhân warning:

- Trong Java, class Throwable (Exception, RuntimeException, Error) implements Serializable. Điều này cho phép exception object có thể serialize.

IDE khuyến cáo bạn khai báo serialVersionUID để:

- Đảm bảo version class ổn định khi serialize.
- Tránh lỗi InvalidClassException khi deserialize exception cũ.

Chapter 14: Đa luồng trong Java

Giới thiệu về đa luồng, một kỹ thuật cho phép chương trình thực thi nhiều tác vụ đồng thời, giúp tận dụng tối đa sức mạnh của CPU và cải thiện hiệu năng đáng kể

#141. Tổng quan về chapter 14

//todo

#142. Giới thiệu Đa luồng (Thread)

1. Đa luồng là gì?

Thread (luồng) là đơn vị nhỏ nhất của CPU có thể thực thi một công việc.

Multithreading (đa luồng) là kỹ thuật cho phép chạy nhiều thread cùng lúc trong một chương trình.

Hiểu đơn giản: Một chương trình Java có thể thực hiện nhiều công việc song song (đa luồng) thay vì tuần tự từng dòng code.

Ví dụ thực tế:

Khi bạn sử dụng máy tính, bạn có thể thực hiện:

- Vừa nghe nhạc (thread 1)
- Vừa lướt web (thread 2)
- Vừa download file (thread 3)

Mỗi công việc chạy trên một thread riêng biệt, nhưng được thực hiện cùng lúc

2. Tại sao cần đa luồng

Lý do	Ý nghĩa
Tận dụng CPU tốt hơn	CPU hiện đại có nhiều core, chạy song song nhiều task nhanh hơn
Tăng tốc chương trình	Thay vì chờ task A xong mới làm task B, có thể chạy A và B cùng lúc
Cải thiện trải nghiệm người dùng	Ví dụ: Ứng dụng vẫn phản hồi UI trong khi đang load dữ liệu

3. Nhược điểm của đa luồng

Dù có nhiều lợi ích, đa luồng cũng có thách thức:

- **Race condition:** Nhiều thread cùng sửa 1 biến → kết quả sai.
- **Deadlock:** Hai thread chờ nhau mãi, không thread nào tiếp tục.
- **Khó debug hơn:** Code chạy song song khó tái hiện lỗi.

Mỗi chương trình Java khi chạy luôn có ít nhất 1 thread, gọi là main thread.

Ở đây, method main chạy trên main thread.

```
public class Demo {  
    public static void main(String[] args) {  
        System.out.println("Hello from main thread!");  
    }  
}
```

#143. Tạo Thread (extends Thread)

1. Thread class trong Java

Thread là 1 class trong package java.lang, giúp bạn tạo thread mới dễ dàng.

Các bước tạo thread bằng extends Thread:

- class MyThread **extends Thread**: tạo class con kế thừa Thread.
- public void **run()**: viết công việc thread sẽ thực hiện.
- t1.**start()**: JVM sẽ gọi run() trong một thread mới.

Lưu ý: start() vs run()

✓ start() : Tạo một thread mới và gọi run() trong thread đó.

✓ run() : Chỉ là 1 method bình thường. Nếu gọi trực tiếp run(), code sẽ chạy trong main thread, **không tạo thread mới**.

2. Minh họa quá trình chạy của các thread khác nhau

//todo: sử dụng Thread.sleep()

#144. Tạo Thread (implements Runnable)

1. Runnable interface là gì?

✓ Runnable là 1 interface trong package java.lang.

✓ Nó có duy nhất một phương thức trừu tượng (abstract method):

```
public interface Runnable {  
    void run();  
}
```

➡ Bạn cần override method run() để định nghĩa công việc cho thread.

Bước 1: Tạo class implements Runnable

Override phương thức run().

Bước 2: Tạo object Runnable

Tạo object của class vừa implements Runnable.

Bước 3: Tạo Thread và truyền Runnable vào constructor

Gọi start() của Thread.

So sánh implements Runnable vs extends Thread

Tiêu chí	implements Runnable	extends Thread
Cách dùng	Implement interface	Kế thừa class Thread
Đa kế thừa	Có thể implements Runnable cùng nhiều interface khác	Không thể extends Thread nếu đã extends class khác
Linh hoạt	Linh hoạt, dễ mở rộng	Giới hạn kế thừa
Thường dùng	✓ Được khuyến nghị	Ít dùng trong dự án lớn

#145. Vòng đời của Thread (Thread Lifecycle)

1.Thread Lifecycle – Vòng đời của một Thread

Một thread/luồng trong Java có thể ở một trong sáu trạng thái sau:

1. **New** (Mới)
2. **Runnable** (Sẵn sàng chạy)
3. **Blocked** (Bị khóa)
4. **Waiting** (Chờ đợi)
5. **Timed Waiting** (Chờ đợi có thời gian)
6. **Terminated** (Kết thúc)

Ví dụ:

```
public class ThreadStateDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            System.out.println("Thread chạy");
            try {
                Thread.sleep(2000); // sleep 2s
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        System.out.println("Trạng thái sau khi tạo: " + t.getState()); // NEW

        t.start();
        System.out.println("Trạng thái sau start: " + t.getState()); // RUNNABLE

        Thread.sleep(100); // chờ 0.1s để đảm bảo thread t sleep
        System.out.println("Trạng thái khi sleep: " + t.getState()); // TIMED_WAITING

        t.join();
        System.out.println("Trạng thái khi kết thúc: " + t.getState()); // TERMINATED
    }
}
```

What's next ? Học gì tiếp theo

Đây là khóa học về Java Core, học các kiến thức nền tảng và cốt lõi của ngôn ngữ Java.

1. Ưu điểm

✓ **Nắm chắc nền tảng Java Core**

- Hiểu rõ cú pháp Java, tư duy lập trình hướng đối tượng OOP (class, inheritance, polymorphism, abstract, interface).
- Thực hành Collection Framework, Generics, Exception Handling, I/O, đa luồng.
- Có tư duy rõ ràng về cách tổ chức code, áp dụng tính trừu tượng và đóng gói.

✓ **Dễ học các công nghệ mới của Java**

- Java Core là nền móng để học Spring, Hibernate, Microservices, hoặc Android.
- Đã có kiến thức vững về syntax và logic, giúp tiếp cận frameworks dễ hơn.

✓ **Tư duy logic khá tốt**

Trong quá trình học core, bạn đã luyện tập problem-solving, hiểu rõ cách vận hành của JVM, Garbage Collector, v.v.

2. Nhược điểm (cần cải thiện)

⚠ **Chưa tiếp cận sâu về design patterns**

Java Core mới giới thiệu cơ bản OOP, chưa áp dụng các mẫu thiết kế (Singleton, Factory, Observer, Strategy...) vào thực tế.

⚠ Chưa hiểu rõ về kiến trúc phần mềm và build tools: Học java core thiên về lý thuyết, code mẫu, cần thực hành nhiều hơn nữa

3. Các định hướng phát triển tiếp theo

Theo định hướng phát triển ứng dụng trên desktop: Java Swing, JavaFX

Theo định hướng phát triển ứng dụng điện thoại di động: Kotlin và Android

Theo định hướng phát triển ứng dụng website: Spring Framework

Về định hướng này, hoidanit có 2 khóa học, bạn có thể tham khảo:

- Khóa học fullstack Java Spring MVC, tham khảo [tại đây](#)
- Khóa học Spring REST, tham khảo [tại đây](#)

Ngoài ra, có thời gian, bạn có thể củng cố thêm tư duy lập trình với:

- Học **Design Patterns** cơ bản
Singleton, Factory, Strategy, Observer, Decorator.
Hiểu và tự code demo từng pattern.
- Luyện **cấu trúc dữ liệu & giải thuật** (Data Structures and Algorithms)
Array, LinkedList, Stack, Queue, Tree, Graph (ở mức cơ bản) + practice LeetCode Easy/Medium.

Đánh Giá (Review/Rating) Khóa Học

1. Lời cảm ơn:

Mình rất hy vọng bạn thấy khóa học này có giá trị, nhưng dù thế nào đi nữa, hãy để lại đánh giá và chia sẻ trải nghiệm của bạn.

Hãy cho mình biết bạn nghĩ gì và viết đánh giá khi bạn có thể.

Mình cũng luôn sẵn sàng trả lời câu hỏi của bạn - hãy gửi tin nhắn trực tiếp cho mình bất cứ lúc nào.

2. Cách đánh giá (review) khóa học, tham khảo chi tiết [tại đây](#)

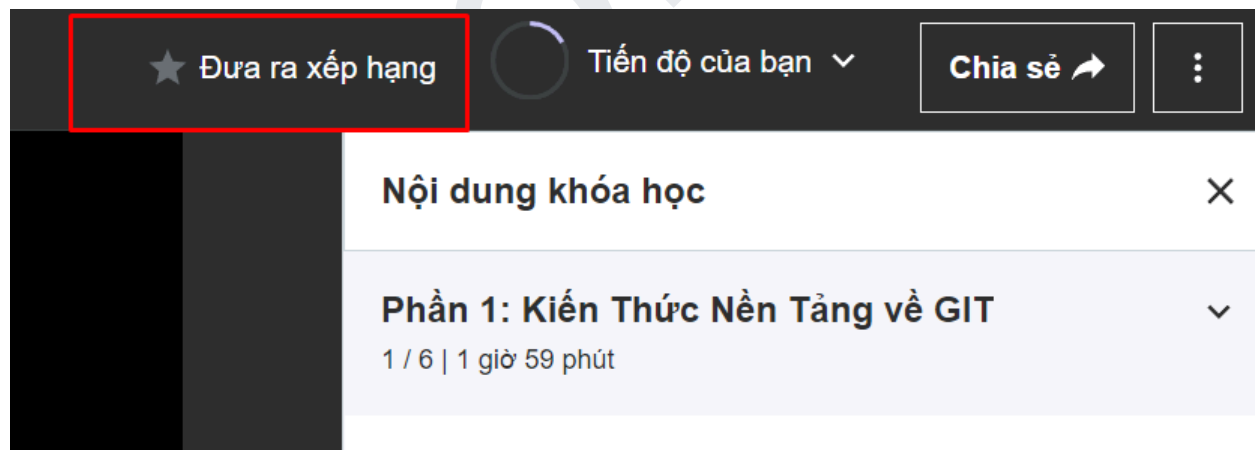
Tạo review đánh giá:

Nhấn vào "đưa ra xếp hạng". Sau đấy rating (star) và viết đánh giá chia sẻ trải nghiệm.

Nếu màn hình của bạn không hiển thị, có 2 nguyên nhân:

1 là bạn đã đánh giá rồi, xem mục 3 bên dưới để biết cách sửa/xóa đánh giá

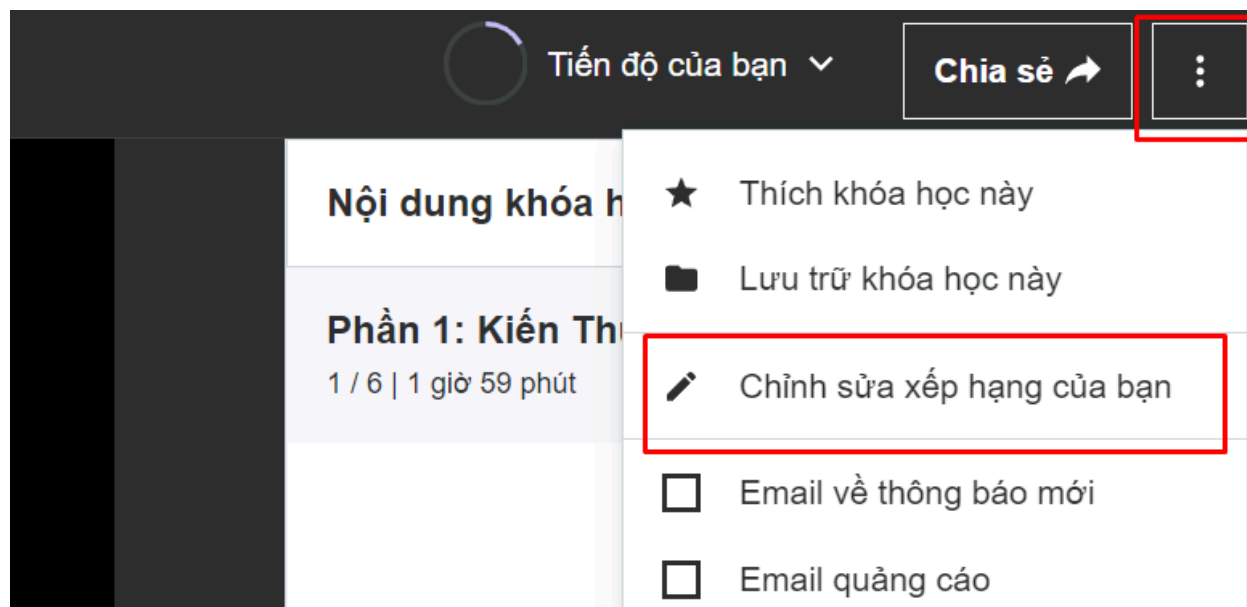
2 là thời gian bạn xem khóa học chưa đủ nên không thể đánh giá



3. Chỉnh sửa/xóa review đã đánh giá:

Bước 1: chọn vào hình 3 dấu chấm tại góc màn hình

Bước 2: chọn chỉnh sửa review



4. Mẹo để Viết Đánh Giá Tốt

Dưới đây là một số điều bạn nên lưu ý khi viết đánh giá cho một khóa học:

Hãy cho chúng tôi biết lý do: Ngoài việc để lại đánh giá sao, vui lòng chia sẻ suy nghĩ của bạn.

Ý kiến của bạn về khóa học rất có giá trị đối với những người học khác, nhưng khi chỉ có đánh giá sao, thật khó để người dùng khác hiểu vì sao bạn lại đưa ra đánh giá đó.

Cụ thể là tốt: Tính cụ thể giúp người học khác quyết định xem khóa học có phù hợp với họ không. Có điểm nào cần cải thiện không?

Khóa học có đáp ứng được kỳ vọng của bạn không?

Phần nào của khóa học khiến bạn thích nhất?

Trung thực: Đánh giá là một trong những yếu tố quan trọng nhất mà mọi người xem xét trước khi đăng ký một khóa học.

Tuy nhiên, đánh giá chỉ có giá trị khi bạn trung thực về cảm nhận của mình về khóa học.

Miễn là bạn chia sẻ cảm nhận một cách tôn trọng, ý kiến phản hồi của bạn rất có giá trị và hữu ích cho cộng đồng học tập của chúng tôi.